

基于策略的强化学习

从 Policy Gradient、REINFORCE、
Actor-Critic 到 PPO

–作者: Yang kunpeng

从参数化策略、轨迹分布与期望回报出发，
推导策略梯度，将理论期望落到 REINFORCE，并进一步用价值函数构造
Actor-Critic 与 PPO。

前言

为什么从策略本身开始。 强化学习最终需要解决的是决策问题：给定当前状态，智能体应该采取什么动作。基于策略的方法直接用参数化分布 $\pi_{\theta}(a | s)$ 描述这种决策规律，再通过调整参数 θ 改变行为。这样一来，强化学习的核心问题可以被表述为：如何定义一个衡量策略优劣的目标函数，以及如何从与环境交互得到的数据中计算这个目标关于策略参数的梯度。

本书前四章的主线。 第一章只讨论基于策略的方法本身，重点建立

$$\pi_{\theta}(a | s) \longrightarrow P_{\theta}(\tau) \longrightarrow J(\theta) \longrightarrow \nabla_{\theta} J(\theta)$$

这条理论链条，并完整推导轨迹形式的 Policy Gradient。第二章再解决一个更实际的问题：理论公式中的期望无法枚举时，怎样利用有限条 trajectory 做 Monte Carlo 估计，并由此得到 REINFORCE。第三章承接 REINFORCE 的高方差与“绝对回报缺少参照”问题，引入 Value、Advantage 与 Critic，并建立 Bootstrapping、TD Error 与 Actor-Critic 的完整关系。第四章从 n -step Advantage 继续，引入 GAE，并围绕旧策略数据复用、Probability Ratio 与 Clipped Surrogate Objective 构造 PPO。

符号约定

符号	含义
θ, ϕ	策略与 Critic 的展平参数向量
s_t	时间步 t 的状态；固定有限时域时含剩余时间
a_t	时间步 t 实际采取的动作
r_t	时间步 t 获得的即时奖励
π_θ	参数为 θ 的策略
$\pi_\theta(a \mid s)$	状态 s 下选择具体动作 a 的概率或概率密度
$\pi_\theta(\cdot \mid s)$	状态 s 下完整的动作条件分布
$P(s', r \mid s, a)$	给定状态-动作后，下一状态与即时奖励的环境联合转移核
μ_0	初始状态分布
τ	一条完整轨迹（trajectory），包含实际采样到的状态、动作与奖励
$P_\theta(\tau)$	策略与环境共同诱导出的轨迹分布
$R(\tau)$	一条轨迹的总回报（return）
G_t	第三、四章中从时间步 t 开始的 discounted reward-to-go； $\gamma = 1$ 时退化为未折扣形式
$G_t^{\gamma=1}$	第二章中未折扣的完整 reward-to-go
$G_t^{[n]}$	第四章中的 n -step bootstrapped target；方括号上标表示步数
$G_t^{(i)}$	第 i 次 rollout 的回报样本；圆括号上标表示样本序号
d_t	真正终止指示；非终止的外部截断取 $d_t = 0$

符号	含义
c_t	GAE 递推连续性指示；片段尾、reset 或真正终止处取零
$V^\pi(s)$	当前策略 π 在状态 s 下的状态价值
$Q^\pi(s, a)$	在状态 s 先固定执行动作 a ，按 π 行动的动作价值
$A^\pi(s, a)$	优势函数， $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$
$V_\phi(s)$	参数为 ϕ 的 Critic 对状态价值的近似
$\rho_t(\theta)$	PPO 中的新旧策略 probability ratio
$\hat{A}_t$	由采样数据得到的 Advantage 估计，常由 GAE 计算
λ	GAE 的衰减参数，控制不同时间尺度 TD error 的组合
$J(\theta)$	策略优化目标，通常取期望回报
γ	价值估计与 GAE 中的折扣因子， $0 < \gamma \leq 1$ ；本书主策略目标保持未折扣 episodic convention

状态、参数与随机变量记号。 本书将策略参数 θ 与 Critic 参数 ϕ 视为展平后的向量；状态 s 和动作 a 则是各自空间中的元素，不预设为向量。对于任务本身规定终止时刻的固定有限时域问题，状态 s_t 包含时间步或剩余时间，因此 $V^\pi(s)$ 、 $Q^\pi(s, a)$ 和 $A^\pi(s, a)$ 不再单独添加时间下标；时间耗尽属于真正终止。外部采样时限不改变任务本身的终止定义。

为沿用强化学习文献的常见记号，本书在采样、期望及条件期望中也使用小写 s_t, a_t, r_t, τ 表示相应随机对象；在给定轨迹或固定训练 batch 时，同样的记号表示已观察到的实现值。随机性由采样分布、条件及上下文明确，而不另作全书大小写迁移。

关于 π_θ 、 $\pi_\theta(a \mid s)$ 与 $\pi_\theta(\cdot \mid s)$ 。 π_θ 表示整个策略对象； $\pi_\theta(a \mid s)$ 是在给定状态 s 时，某一个具体动作 a 对应的概率或概率密度； $\pi_\theta(\cdot \mid s)$ 则表示固定 s 后的整个动作分布。符号“ $\cdot$ ”表示该位置暂不指定具体动作。

关于 $\tau \sim \pi_\theta$ 。强化学习文献中经常用 $\tau \sim \pi_\theta$ 表示“使用策略 π_θ 在给定环境中 rollout 得到轨迹”。严格地说，轨迹服从的是策略与环境共同诱导出的分布 $P_\theta(\tau)$ 。本书在严格推导中使用 $\tau \sim P_\theta$ ，在算法性文字中允许使用 $\tau \sim \pi_\theta$ 这一惯用简写。

目录

前言	ii
符号约定	iii
第一章 基于策略的强化学习	1
1.1 参数化策略与轨迹分布	1
1.2 期望回报与策略优化目标	3
1.3 从轨迹分布到 Policy Gradient	4
1.4 策略梯度的直觉与性质	7
第二章 REINFORCE：蒙特卡洛策略梯度	10
2.1 从期望梯度到采样估计	10
2.2 REINFORCE 算法	11
2.3 Reward-to-Go 与因果结构	12
2.4 REINFORCE 的统计特性与局限	14
第三章 Actor-Critic：价值函数辅助策略学习	16
3.1 从 Baseline 到价值函数与 Advantage	16
3.2 Critic 与 Policy Evaluation	18
3.3 Bootstrapping、TD Error 与 Advantage 估计	20
3.4 Actor-Critic 算法与实现	22
3.5 Actor-Critic 的统计特性与局限	24
第四章 PPO：近端策略优化	26
4.1 新旧策略的概率比	26
4.2 裁剪目标与近端策略更新	28
4.3 广义优势估计	29
4.4 PPO 算法	32
附录 A 附录	34

A.1	Policy Gradient 的严格推导	34
A.2	REINFORCE 中的若干关键推导	38
A.3	Actor-Critic 中的关键推导	42
A.4	PPO 中的关键推导	46

基于策略的强化学习

基于策略的强化学习把“如何行动”本身作为需要学习的对象。设智能体在时间步 t 观察到状态 s_t ，策略用条件分布 $\pi_{\theta}(a | s)$ 描述在状态 s 下选择动作 a 的规律，其中 θ 是将全部可学习参数展平得到的参数向量。若策略由神经网络实现，那么训练的目标就是调整网络参数，使策略在与环境交互时能够获得更大的长期回报。

本章不从某个具体算法出发，而是先解决一个更基础的问题：一个**参数化策略为什么可以通过梯度来优化**？为回答这个问题，需要依次建立策略、轨迹分布、期望回报以及策略梯度之间的关系。

1.1 参数化策略与轨迹分布

策略是状态到动作分布的映射。 随机策略可以写成 $\pi_{\theta}(\cdot | s)$ ，它表示给定状态 s 后，策略在动作空间 $\mathcal{A}$ 上定义的完整条件分布。当指定一个具体动作 a 时， $\pi_{\theta}(a | s)$ 就是该动作对应的概率；若动作空间连续，则它表示相应的概率密度值。

在离散动作空间中，神经网络通常先输出一组 logits，再经过 Softmax 得到动作分布。若某个状态下有三个动作，并得到

$$\pi_{\theta}(\cdot | s) = (0.2, 0.5, 0.3),$$

在本例中， a_2 的下标表示动作类别编号，不是时间步；选择第二个动作的概率就是

$$\pi_{\theta}(a_2 | s) = 0.5.$$

动作本身通过采样得到：

$$a_t \sim \pi_{\theta}(\cdot | s_t).$$

连续动作也可以直接由参数化分布表示。例如，对一维连续动作，一个常见形式是由神经网络输出 Gaussian 的均值和标准差：

$$a_t \sim \mathcal{N}(\mu_{\theta}(s_t), \sigma_{\theta}^2(s_t)).$$

策略通过环境形成轨迹。 动作被执行后，环境根据当前状态和动作共同产生即时奖励与下一个状态。用联合转移核表示为

$$(s_{t+1}, r_t) \sim P(\cdot, \cdot \mid s_t, a_t).$$

于是一次交互由两种机制交替组成：

$$s_t \xrightarrow{\pi_{\theta}} a_t \xrightarrow{\text{environment}} (r_t, s_{t+1}).$$

把多个时间步连起来，就得到一条轨迹

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_{T-1}, a_{T-1}, r_{T-1}, s_T).$$

这里把实际获得的奖励也纳入 τ 。这样即使给定 (s_t, a_t) 后 reward 仍具有随机性，一旦整条 τ 已经给定，轨迹回报 $R(\tau)$ 仍是确定的函数值。

这里需要区分两个概率对象。 $\pi_{\theta}(a_t \mid s_t)$ 只描述某一步怎样选择动作，而整条轨迹的出现概率还取决于初始状态以及环境如何转移。因此，轨迹并不“只由 policy 产生”。

轨迹的联合概率。 设初始状态来自分布 μ_0 。利用条件概率链式法则和 Markov 结构，一条轨迹的联合概率可以写为

$$P_{\theta}(\tau) = \mu_0(s_0) \prod_{t=0}^{T-1} \pi_{\theta}(a_t \mid s_t) P(s_{t+1}, r_t \mid s_t, a_t). \quad (1.1)$$

数学基础提示：式 (1.1) 的联合概率分解与严格条件，详见附录 A.1。

式 (1.1) 中有三类因素：

$$\underbrace{\mu_0(s_0)}_{\text{初始状态}} \quad \underbrace{\pi_{\theta}(a_t \mid s_t)}_{\text{策略}} \quad \underbrace{P(s_{t+1}, r_t \mid s_t, a_t)}_{\text{环境}}.$$

策略负责“在当前状态选择什么动作”，环境负责“执行动作后即时奖励与下一状态如何产生”。二者反复交替，才共同决定一条完整轨迹。

为什么写成 $P_{\theta}(\tau)$ 。 环境联合转移核 $P(s_{t+1}, r_t | s_t, a_t)$ 通常不含策略参数 θ ，但轨迹概率中包含每个时间步的 $\pi_{\theta}(a_t | s_t)$ 。因此，只要改变 θ ，动作分布就会变化，进而改变哪些状态更容易被访问以及哪些轨迹更容易出现。于是整条轨迹的分布也依赖 θ ，记为 $P_{\theta}(\tau)$ 。

关于 $\tau \sim \pi_{\theta}$ 的惯用写法。 文献中常把“使用策略 π_{θ} 在环境中 rollout 一条轨迹”简写为 $\tau \sim \pi_{\theta}$ 。这并不表示轨迹概率在数学上等于策略分布；更严格地说，轨迹服从由策略和环境共同诱导的分布 P_{θ} 。

1.2 期望回报与策略优化目标

有了轨迹分布之后，下一步需要回答：**什么样的策略更好？** 强化学习通过 reward 为交互结果赋予数值反馈，再用一条轨迹上的累计 reward 衡量该轨迹的整体表现。

即时奖励与轨迹回报。 设时间步 t 获得即时奖励 r_t 。在有限时域问题中，可以把一条轨迹的总回报写成

$$R(\tau) = \sum_{t=0}^{T-1} r_t. \quad (1.2)$$

如果希望较远未来的奖励权重更小，也可以定义折扣回报

$$R_{\gamma}(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t, \quad 0 < \gamma \leq 1.$$

本书的主策略目标采用式 (1.2) 的有限时域、未折扣 episodic return。后续 Actor-Critic 与 GAE 会引入 $\gamma < 1$ 的 discounted reward-to-go 作为价值估计与偏差-方差折中的算法性权重；除非另行说明，它不被当作对本章未折扣目标的完全等价改写。

为什么优化期望而不是某一条轨迹。 同一个策略可以因为动作采样和环境随机性产生不同轨迹。某一次获得高回报，可能只是偶然；某一次失败，也不能说明整个策略一定很差。因此，策略本身的好坏应该由其在轨迹分布上的平均表现衡量。定义

$$J(\theta) = \mathbb{E}_{\tau \sim P_{\theta}}[R(\tau)].$$

$J(\theta)$ 称为策略目标函数 (objective function)。它表示固定参数 θ 后, 反复使用当前策略与环境交互时能够获得的期望回报。

在离散轨迹空间中, 期望可以展开为

$$J(\theta) = \sum_{\tau} P_{\theta}(\tau) R(\tau). \quad (1.3)$$

也就是说, 每一条可能轨迹的回报乘以它出现的概率, 再对所有轨迹求和。连续状态或动作空间中, 相应的求和会变成积分, 但推导结构相同。

策略学习变成参数优化。 有了 $J(\theta)$, 基于策略的强化学习可以写成一个明确的优化问题:

$$\theta^* = \arg \max_{\theta} J(\theta).$$

如果能够计算 $J(\theta)$ 关于参数的梯度, 就可以采用梯度上升:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta), \quad (1.4)$$

其中 $\alpha > 0$ 是学习率。式 (1.4) 的意义非常直接: 沿着使期望回报增长最快的局部方向调整策略参数。

真正的困难在于, $J(\theta)$ 不是一个普通的监督学习损失。参数 θ 通过策略改变动作分布, 动作又影响后续状态, 从而改变整条轨迹的分布。因此, 要计算 $\nabla_{\theta} J(\theta)$, 必须处理“概率分布本身依赖参数”这一结构。

1.3 从轨迹分布到 Policy Gradient

本节从式 (1.3) 出发, 推导轨迹形式的 Policy Gradient。推导的核心不是对环境求导, 而是把“轨迹概率对参数的导数”转换为“策略 log probability 的导数”。

第一步: 直接对策略目标求梯度。 由

$$J(\theta) = \sum_{\tau} P_{\theta}(\tau) R(\tau),$$

在附录 A.1 声明的可微性、共同支持集与求导交换条件下，对 θ 求梯度：

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \nabla_{\theta} \sum_{\tau} P_{\theta}(\tau) R(\tau) \\ &= \sum_{\tau} R(\tau) \nabla_{\theta} P_{\theta}(\tau).\end{aligned}\tag{1.5}$$

这里把 $R(\tau)$ 看作给定轨迹后的固定评价：一旦 τ 已经确定，reward 由环境和任务定义，不再显式依赖待优化参数 θ 。参数的影响通过“这条轨迹出现的概率”进入目标函数。

式 (1.5) 中最难处理的是

$$\nabla_{\theta} P_{\theta}(\tau).$$

轨迹概率本身是一长串概率的乘积，直接展开乘积求导并不方便。

第二步：使用 log-derivative trick。 对于任意正的可微函数 f_{θ} ，有

$$\nabla_{\theta} \log f_{\theta} = \frac{\nabla_{\theta} f_{\theta}}{f_{\theta}}.$$

移项得到

$$\nabla_{\theta} f_{\theta} = f_{\theta} \nabla_{\theta} \log f_{\theta}.$$

令 $f_{\theta} = P_{\theta}(\tau)$ ，则

$$\nabla_{\theta} P_{\theta}(\tau) = P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau).\tag{1.6}$$

数学基础提示：从 log-derivative trick 到轨迹形式 Policy Gradient 的完整推导，详见附录 A.1。

把式 (1.6) 代回式 (1.5)：

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \sum_{\tau} P_{\theta}(\tau) R(\tau) \nabla_{\theta} \log P_{\theta}(\tau) \\ &= \mathbb{E}_{\tau \sim P_{\theta}} [R(\tau) \nabla_{\theta} \log P_{\theta}(\tau)].\end{aligned}\tag{1.7}$$

到这里已经发生了一个重要变化：原来难处理的 $\nabla_{\theta} P_{\theta}(\tau)$ 被改写成了 log probability 的梯度。

第三步：展开轨迹 log probability。 由式 (1.1):

$$P_{\theta}(\tau) = \mu_0(s_0) \prod_{t=0}^{T-1} \pi_{\theta}(a_t | s_t) P(s_{t+1}, r_t | s_t, a_t).$$

两边取对数，乘积变成求和：

$$\begin{aligned} \log P_{\theta}(\tau) &= \log \mu_0(s_0) \\ &\quad + \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t | s_t) \\ &\quad + \sum_{t=0}^{T-1} \log P(s_{t+1}, r_t | s_t, a_t). \end{aligned} \tag{1.8}$$

这一变形看似只是代数操作，却是整个推导的关键。轨迹概率原本把策略和环境混合在一个大乘积中，取对数后它们变成了彼此独立的加法项。

第四步：对 θ 求梯度，环境项消失。 对式 (1.8) 求梯度：

$$\begin{aligned} \nabla_{\theta} \log P_{\theta}(\tau) &= \nabla_{\theta} \log \mu_0(s_0) \\ &\quad + \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \\ &\quad + \sum_t \nabla_{\theta} \log P(s_{t+1}, r_t | s_t, a_t). \end{aligned}$$

在标准强化学习建模中， θ 是策略参数，而初始状态分布 μ_0 与环境状态转移 P 是外部给定的。因此

$$\nabla_{\theta} \log \mu_0(s_0) = 0,$$

以及

$$\nabla_{\theta} \log P(s_{t+1}, r_t | s_t, a_t) = 0.$$

于是只剩策略项：

$$\nabla_{\theta} \log P_{\theta}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t). \tag{1.9}$$

这个结果说明，虽然轨迹是由策略和环境共同产生的，但当我们只对策略参数 θ 求梯度时，环境概率本身不需要可微，也不需要显式求导。环境仍然通过“实际采样到了哪些状态和 reward”影响训练，但它不出现在策略参数的解析梯度中。

第五步：得到轨迹形式的 Policy Gradient。 把式 (1.9) 代入式 (1.7)：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim P_{\theta}} \left[R(\tau) \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right].$$

也可以把 $R(\tau)$ 放入求和：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim P_{\theta}} \left[\sum_{t=0}^{T-1} R(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]. \quad (1.10)$$

这就是本章需要建立的核心理论结果。

关于“Policy Gradient Theorem”这个名称。 强化学习文献中更经典的 Policy Gradient Theorem 常写成状态访问分布与 $Q^{\pi}(s, a)$ 的形式。本章采用的是从 trajectory likelihood 出发的 likelihood-ratio 推导，它直接通向 REINFORCE，也更适合解释后续 LLM 强化学习中常见的 log-probability 目标。二者并不矛盾，只是组织期望的方式不同。

1.4 策略梯度的直觉与性质

式 (1.10) 给出了轨迹形式的策略梯度：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim P_{\theta}} \left[\sum_{t=0}^{T-1} R(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right].$$

理解这个式子的关键，是看清 $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ 与 $R(\tau)$ 各自的作用。

log probability 的梯度决定动作概率如何变化。 对于已经采样到的状态-动作对 (s_t, a_t) ， $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ 给出了使该动作 log probability 局部增大的参数方向。沿这个方向做足够小的更新，会倾向于提高当前状态下动作 a_t 的概率；沿相反方向更新，则会倾向于降低它的概率。

这个梯度本身并不知道动作 a_t 是好是坏，它只描述参数应当怎样变化。

回报为这个方向提供训练信号。 策略梯度中真正使用的是 $R(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ 。因此，log probability 的梯度给出参数变化方向，而回报决定这次采样结果对更新产生多大影响。

如果某类动作经常出现在高回报轨迹中，经过多次采样和更新后，策略会逐渐提高这类动作出现的概率；反之，其概率会受到抑制。这里强调的是多次采样后的统计效果，而不是由一条轨迹准确判断每个动作的好坏。

轨迹回报带来了粗粒度的信用分配。 在式 (1.10) 中，同一条轨迹里的所有时间步都乘以同一个 $R(\tau)$ 。因此，最原始的轨迹级策略梯度只能判断“整条轨迹表现如何”，却不能区分哪些动作真正带来了好的结果。

对于一条较长的轨迹，其中可能既包含关键的正确决策，也包含影响很小甚至并不理想的动作。若所有动作都使用相同的轨迹回报作为权重，训练信号显然比较粗糙。

因此，如何把回报更合理地分配到不同时间步，是后续改进的核心。下一章中的 Reward-to-Go、baseline 和 advantage，都可以看作对这一问题的逐步处理。

单条轨迹只是策略梯度的一个样本。 式 (1.10) 中真正需要计算的是轨迹分布下的期望 $\mathbb{E}_{\tau \sim P_{\theta}}[\dots]$ 。实际训练时无法枚举所有轨迹，只能从当前策略中采样有限样本，并用样本平均近似这个期望。

因此，单条轨迹对应的更新具有较大随机性，而策略梯度描述的是多次采样后在期望意义下的平均更新方向。这也是从策略梯度公式走向 REINFORCE 的关键一步。

环境不需要参与反向传播。 由上一节已经得到

$$\nabla_{\theta} \log P_{\theta}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

环境转移概率 $P(s_{t+1}, r_t | s_t, a_t)$ 不会出现在关于 θ 的最终梯度中。

因此，策略梯度只要求能够与环境交互并获得状态转移和 reward，而不要求环境本身可微。真正参与反向传播的是策略模型的 $\log \pi_{\theta}(a_t | s_t)$ 。

从策略梯度到 REINFORCE。 到这里，策略梯度的基本结构已经清楚：策略产生动作，环境给出结果，回报为已采样动作的 $\log \text{probability}$ 梯度提供训练信号。

接下来只需考虑如何利用有限条采样轨迹近似

$$\mathbb{E}_{\tau \sim P_{\theta}}[\dots],$$

并据此完成参数更新。最直接的方法就是 Monte Carlo 估计，由此得到下一章的 REINFORCE。

本章主线可以压缩为：参数化策略 π_θ 决定动作分布，策略与环境共同诱导轨迹分布 $P_\theta(\tau)$ ，轨迹回报定义目标 $J(\theta)$ ，log-derivative trick 再把 $\nabla_\theta J(\theta)$ 转换成可由策略 log probability 表达的梯度。

REINFORCE：蒙特卡洛策略梯度

第一章得到轨迹形式的策略梯度

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim P_{\theta}} \left[R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]. \quad (2.1)$$

这个公式已经说明 reward 如何改变 policy，但它仍然包含对所有可能轨迹的期望。REINFORCE 的核心工作，就是用当前策略实际采样得到的有限条 trajectory 近似这个期望，并把估计出的梯度用于更新策略参数。

2.1 从期望梯度到采样估计

理论期望为什么不能直接计算。 若每个时间步有多个可能动作，轨迹数量会随着 horizon 快速增长；环境状态又可能连续，使完整轨迹空间甚至不可数。因此，式 (2.1) 中的期望通常无法通过枚举精确计算。

实际可行的办法是使用当前策略与环境交互，采样 m 条轨迹：

$$\tau^{(1)}, \tau^{(2)}, \dots, \tau^{(m)} \sim P_{\theta}.$$

对于任意随机变量 X ，其期望可以由样本均值近似：

$$\mathbb{E}[X] \approx \frac{1}{m} \sum_{i=1}^m X^{(i)}.$$

因此，策略梯度可以估计为

$$\hat{g} = \frac{1}{m} \sum_{i=1}^m \sum_{t=0}^{T_i-1} R(\tau^{(i)}) \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}). \quad (2.2)$$

这里 T_i 允许不同 episode 长度不同。

数学基础提示：Monte Carlo 策略梯度估计的无偏性与多轨迹平均的方差，详见附录 A.2。

两层求和分别表示什么。 式 (2.2) 中，外层 $\sum_{i=1}^m$ 在不同 trajectory 之间求和，内层 $\sum_t$ 在同一条 trajectory 的不同时间步之间求和。符号 $a_t^{(i)}$ 和 $s_t^{(i)}$ 分别表示第 i 条轨迹在时间步 t 的动作和状态。

例如，若 $m = 2$ ，第一条轨迹回报为 10，第二条轨迹回报为 -4，则估计梯度具有形式

$$\hat{g} = \frac{1}{2} \left(10 \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t^{(1)} | s_t^{(1)}) - 4 \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t^{(2)} | s_t^{(2)}) \right).$$

第一条高回报轨迹倾向于增加其中动作的概率；第二条负回报轨迹则倾向于降低对应动作的概率。多条 trajectory 的贡献平均后，得到本轮训练使用的更新方向。

真实梯度与估计梯度。 需要严格区分理论梯度 $\nabla_{\theta} J(\theta)$ 和有限样本估计 $\hat{g}$ 。前者是在当前策略分布下定义的理论期望，是一个确定的数学量；后者依赖本轮随机采到的有限数据，因此本身是随机变量。所谓“策略梯度方差大”，更准确地说，是 $\hat{g}$ 作为梯度估计器的方差可能很大，而不是理论梯度 $\nabla_{\theta} J(\theta)$ 自己在随机变化。

在适当正则条件下，式 (2.2) 对应一个无偏 Monte Carlo 估计：

$$\mathbb{E}[\hat{g}] = \nabla_{\theta} J(\theta).$$

但“无偏”不代表单次估计准确。即使平均意义上方向正确，不同 batch 得到的 $\hat{g}$ 仍可能相差很大。

2.2 REINFORCE 算法

有了式 (2.2)，REINFORCE 只需将这个 Monte Carlo 梯度估计器用于参数更新。

从梯度到参数更新。 给定估计梯度 $\hat{g}$ ，梯度上升更新为

$$\theta \leftarrow \theta + \alpha \hat{g}. \quad (2.3)$$

将式 (2.2) 代入，可以看到算法只需要从采样数据中记录状态 $s_t^{(i)}$ 、动作 $a_t^{(i)}$ 与轨迹回报 $R(\tau^{(i)})$ 。环境联合转移核 $P(s_{t+1}, r_t | s_t, a_t)$ 并不需要显式计算。

REINFORCE 的基本过程。 一个 batch 版本可以概括为：

1. 用当前策略 π_{θ} 与环境交互，采样 m 个完整 episode；
2. 对每个 episode 计算总回报 $R(\tau^{(i)})$ ；
3. 对 episode 中每个已采样动作计算 $\log \pi_{\theta}(a_t^{(i)} | s_t^{(i)})$ ；
4. 按式 (2.2) 聚合这些项，得到 $\hat{g}$ ；
5. 按式 (2.3) 更新参数，再用新策略重新采样。

这里“重新采样”非常重要，因为 REINFORCE 使用的是当前策略分布下的期望。策略变化后，下一轮理论目标对应的轨迹分布也随之变化。

从梯度上升写成深度学习中的 loss。 PyTorch、JAX 等框架默认通过最小化 loss 进行梯度下降。为了得到与梯度上升相同的方向，可以定义

$$\mathcal{L}_{\text{REINFORCE}}(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_t R(\tau^{(i)}) \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}). \quad (2.4)$$

因为

$$-\nabla_{\theta} \mathcal{L}_{\text{REINFORCE}} = \hat{g},$$

所以最小化式 (2.4) 等价于沿 Monte Carlo 策略梯度方向最大化回报。

为什么只对已采样动作取 log probability。 在一个时间步，策略可能对许多动作都赋予概率，但环境真正执行的只有采样到的 a_t 。策略梯度使用 $\log \pi_{\theta}(a_t | s_t)$ 而不是整个概率向量，没有被采样的动作会通过 Softmax 等参数共享结构间接受影响，但不会直接出现在该条 trajectory 的 log-likelihood 中。

2.3 Reward-to-Go 与因果结构

基础 REINFORCE 把整条轨迹的同一个 $R(\tau)$ 乘到所有时间步上。这个形式在数学上成立，但从因果关系看过于粗糙：时间步 t 的动作不可能影响它发生之前已经获得的奖励。

从整条轨迹回报到未来回报。先考虑有限时域、无折扣形式

$$R(\tau) = \sum_{k=0}^{T-1} r_k.$$

对时间步 t 而言，可以把总回报拆成

$$R(\tau) = \underbrace{\sum_{k=0}^{t-1} r_k}_{\text{过去奖励}} + \underbrace{\sum_{k=t}^{T-1} r_k}_{\text{未来奖励}}.$$

策略梯度中的第 t 项是 $R(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ 。过去奖励在动作 a_t 被采样之前已经发生，因此它们与“当前动作随机性”的 score 项在条件期望下没有贡献。其关键恒等式是

$$\mathbb{E}_{a \sim \pi_{\theta}(\cdot | s)} [\nabla_{\theta} \log \pi_{\theta}(a | s)] = 0.$$

因此，在不改变期望梯度的前提下，可以去掉动作发生之前的 reward，只保留从当前时间步开始的 reward-to-go。

数学基础提示：score function 零均值以及 Reward-to-Go 不改变期望梯度的严格证明，详见附录 A.2。

对本书第一章采用的未折扣 episodic 目标，严格等价的 reward-to-go 定义为

$$G_t^{\gamma=1} = \sum_{k=t}^{T-1} r_k.$$

Reward-to-Go 形式的 REINFORCE。 使用 $G_t^{\gamma=1}$ 时，策略梯度的 Monte Carlo 估计可写为

$$\hat{g} = \frac{1}{m} \sum_{i=1}^m \sum_t (G_t^{\gamma=1})^{(i)} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}).$$

相应的 loss 为

$$\mathcal{L}_{\text{RTG}}(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_t (G_t^{\gamma=1})^{(i)} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}).$$

这一步与原始未折扣目标的期望梯度严格等价，因为它只去掉了动作发生之前、在条件期望中贡献为零的 reward。

引入折扣后的算法性记号。实践中的 Actor-Critic 与 GAE 常定义

$$G_t^\gamma = \sum_{k=t}^{T-1} \gamma^{k-t} r_k, \quad 0 < \gamma \leq 1.$$

当 $\gamma = 1$ 时, $G_t^\gamma = G_t^{\gamma=1}$ 。当 $\gamma < 1$ 时, 在本书保持第一章未折扣主目标的 convention 下, 直接用 G_t^γ 替代 $G_t^{\gamma=1}$ 应理解为更强调近期奖励的 discounted approximation / bias-variance tradeoff, 而不是与原始 $\nabla_{\theta} J(\theta)$ 完全相同的无偏改写。第三、四章为简洁起见把 G_t^γ 简记为 G_t 。若另行把策略目标本身定义为从时间零开始的折扣回报, 则还会出现 $\gamma^t G_t^\gamma$ 的严格权重关系, 详见附录“折扣 Reward-to-Go 的记号细节”。

Reward-to-Go 改善了什么。 如果轨迹前半段已经获得某些 reward, 那么后面动作不应该因为这些过去 reward 被重复强化或惩罚。Reward-to-Go 去掉了这种无因果贡献的噪声, 因此通常比“所有时间步都乘整条 episode return”的估计方差更低。

但 Reward-to-Go 仍然没有完全解决 credit assignment。即便只考虑未来 reward, 一个时间步之后通常仍包含多个动作, 因此 G_t 依然只是一个较粗粒度的训练信号。

2.4 REINFORCE 的统计特性与局限

REINFORCE 的价值在于把策略梯度理论变成一个直接可执行的 Monte Carlo 算法。与此同时, 它的几个局限也来自同一套采样机制。

梯度估计方差较大。 $\hat{g}$ 依赖随机采样的 trajectory。即使参数 θ 完全相同, 不同 episode 也可能因为动作采样、初始状态、环境随机性以及长时间累积效应得到非常不同的 return, 因此不同 batch 得到的梯度估计可能在方向和模长上明显波动。

这种现象并不表示理论梯度本身在随机变化。 $\nabla_{\theta} J(\theta)$ 是当前策略分布下的期望, 而随机的是有限样本估计 $\hat{g}$ 。增加 trajectory 数量可以通过样本平均降低方差, 但代价是更多环境交互。此外, 原始 REINFORCE 直接使用 return 作为权重, 没有状态相关的参考水平, 这也会带来额外波动。

Credit Assignment 仍然粗糙。 Reward-to-Go 去掉了当前动作不可能影响的过去 reward, 但仍然把整个未来累计回报 G_t 作为当前动作的权重。例如

$$a_t \rightarrow a_{t+1} \rightarrow \cdots \rightarrow a_{T-1} \rightarrow \text{success},$$

最终成功究竟主要由哪个动作造成， G_t 本身仍不能区分。长时域任务中，这种粗粒度的信用分配会进一步放大梯度估计的噪声。

样本效率有限。 REINFORCE 需要先 rollout，再根据当前策略采到的数据更新参数。策略改变后，新的目标对应新的轨迹分布，因此旧数据不能像静态监督数据那样长期、反复地直接复用。其核心是

$$\text{data distribution depends on } \pi_{\theta}.$$

因此，REINFORCE 通常需要持续使用当前策略重新采样，环境交互成本可能较高。

REINFORCE 的本质是：用当前策略采样 trajectory，用 Monte Carlo return 加权已采样动作的 log probability，再通过梯度下降最小化负的加权 log-likelihood。Reward-to-Go 可以去掉与当前动作无因果关系的过去 reward，但梯度方差、粗粒度 credit assignment 和 on-policy 数据依赖仍然存在。下一章将在此基础上引入价值估计，讨论 Actor-Critic 方法。

Actor-Critic: 价值函数辅助策略学习

第二章最后留下了一个直接的问题：Reward-to-Go G_t 虽然比整条轨迹回报更符合因果结构，但它仍然只是一次随机 rollout 得到的绝对回报。若某次 $G_t = 100$ ，仅凭这个数值无法判断当前动作究竟“好多少”；真正需要的参照是，在同一个状态下，当前策略通常能够取得怎样的回报。

Actor-Critic 的基本思想正是为策略引入这样一个可学习的评价器。Actor 仍然是参数化策略 π_θ ，负责选择动作；Critic 则学习当前策略的价值函数，用来评价状态以及当前动作相对于正常水平的好坏。这样，Policy Gradient 不再只能依赖高方差的单次 Monte Carlo return，而可以利用价值函数构造更稳定的更新信号。

从本章开始，若无特别说明， G_t 表示第二章定义的 discounted reward-to-go G_t^γ ，并相应定义 V^π, Q^π, A^π ；为避免符号过重，不再把 γ 写进这些函数的下标。当 $\gamma = 1$ 时，它们与第一章未折扣 episodic 目标完全一致；当 $\gamma < 1$ 时，下面的 Actor 更新方向应理解为对该主目标的 discounted approximation，而非声称仍是完全相同的无偏梯度。

本章沿着

$$G_t \longrightarrow V^\pi(s), Q^\pi(s, a) \longrightarrow A^\pi(s, a) \longrightarrow \text{Critic} \longrightarrow \delta_t \longrightarrow \text{Actor update}$$

这条主线展开。

3.1 从 Baseline 到价值函数与 Advantage

为什么需要状态相关的参照。 第二章已经证明，只要某个 baseline 在给定状态和历史后不依赖当前采样动作 a_t ，那么把它从策略梯度的回报权重中减去，不会改变期望梯度。附录 A.2 的式 (A.10) 给出了这一零均值性质。

因此，与其直接使用 $G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ ，更自然的想法是寻找一个依赖状态 s_t 的参照值 $b(s_t)$ ，使用

$$(G_t - b(s_t)) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t). \quad (3.1)$$

如果 $b(s_t)$ 能表示“从这个状态出发通常能取得多少回报”，那么式 (3.1) 就不再只看绝对回报，而是在比较本次结果与当前状态下的正常水平。

状态价值函数。 沿用符号约定，固定有限时域时状态包含剩余时间。对当前策略 π ，定义状态价值函数

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t | s_t = s].$$

它回答的是：**已经位于状态 s ，接下来按照策略 π 正常行动时，未来 reward-to-go 的平均值是多少**

这里的“平均”非常重要。一次 rollout 只会得到某个具体的 G_t ； $V^{\pi}(s)$ 则是把从同一个状态出发、按照同一策略继续行动时所有可能未来的回报做条件平均。因此 $V^{\pi}(s)$ 不是某一次轨迹的 **realized return**，而是当前策略在状态 s 上的期望表现。

动作价值函数。 定义动作价值函数

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | s_t = s, a_t = a].$$

它与 V^{π} 的区别只在当前这一步：在 $Q^{\pi}(s, a)$ 中，**当前动作 $a_t = a$ 已经固定**；动作执行完以后，从下一时间步开始仍然按照策略 π 行动。

可以把两者写成两条不同的决策链：

$$V^{\pi}(s_t) : \quad s_t \xrightarrow{a_t \sim \pi(\cdot | s_t)} s_{t+1} \xrightarrow{\pi} \dots,$$

而

$$Q^{\pi}(s_t, a_t) : \quad s_t \xrightarrow{\text{固定 } a_t} s_{t+1} \xrightarrow{\pi} \dots.$$

因此 $Q^{\pi}(s_t, a_t)$ 不是“以后每一步都固定做 a_t ”，而只是固定当前第一步。

V^{π} 与 Q^{π} 的关系。 在状态 s 下，如果当前动作也按照 π 采样，那么状态价值就是各个动作价值按策略概率加权后的平均：

$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi(\cdot | s)} [Q^{\pi}(s, a)].$$

离散动作空间中，它写成

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) Q^\pi(s, a).$$

因此， $V^\pi(s)$ 可以理解为“当前策略在状态 s 下的平均动作水平”，而 $Q^\pi(s, a)$ 表示明确选择某个动作 a 后的平均未来表现。

优势函数。 有了状态平均水平，就可以定义动作相对于这个平均水平的优势：

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

若 $A^\pi(s, a) > 0$ ，表示动作 a 比当前策略在状态 s 下的正常水平更好；若 $A^\pi(s, a) < 0$ ，则表示它低于正常水平。

例如，若 $V^\pi(s) = 10$ 、 $Q^\pi(s, a) = 14$ ，则 $A^\pi(s, a) = 4$ 。这个“4”比单独看到 $Q^\pi(s, a) = 14$ 更有策略更新意义，因为它明确告诉 Actor：这个动作比当前策略在该状态下的平均选择好 4 个单位。

从 REINFORCE 到 Advantage 形式。 沿用上述 reward-to-go 记号，可以把 Actor 使用的 Advantage 加权方向写成

$$\mathbf{g}_{\text{AC}}(\boldsymbol{\theta}) = \mathbb{E}_\pi \left[\sum_t A^\pi(s_t, a_t) \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t | s_t) \right]. \quad (3.2)$$

当 $\gamma = 1$ 时， $\mathbf{g}_{\text{AC}}(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ ；当 $\gamma < 1$ 时，本书把它作为 discounted Actor 更新方向使用。从 G_t 到 Q^π 再到 A^π 的条件期望替换本身仍是严格的，推导放在附录 A.3。

数学基础提示：从 Reward-to-Go 到 Q^π 与 Advantage 形式的严格推导，详见附录 A.3。

式 (3.2) 给出了 Actor-Critic 的核心目标：Actor 仍然按照 Policy Gradient 更新，但现在真正需要估计的是 $A^\pi(s_t, a_t)$ ，而不再只是直接使用一次采样得到的 G_t 。

3.2 Critic 与 Policy Evaluation

理论上的 $V^\pi(s)$ 是一个条件期望，但实际环境中通常没有它的解析表达式。因此 Actor-Critic 引入一个参数化函数

$$V_\phi(s) \approx V^\pi(s),$$

其中 ϕ 是将 Critic 的全部可学习参数展平得到的参数向量。若使用神经网络实现，输入是状态 s ，输出是一个标量价值预测。

Policy Evaluation 的含义。 这里的 Policy Evaluation 不是寻找最优策略，而是：**固定当前策略 π ，估计这个策略自己的价值函数 V^π 。** Actor 改变策略以后，被评价的对象也随之改变，因此 Critic 需要随着当前策略持续更新。

最直接的 Monte Carlo 拟合。 第二章已经能够从完整 episode 中计算

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k.$$

根据状态价值的定义， G_t 是 $V^\pi(s_t)$ 的一次随机样本。因此可以收集训练对 (s_t, G_t) ，并用回归训练 Critic。以下用 N 表示训练 batch 中有效 transition 的数量，用 i 对展平后的训练样本编号； s_i, G_i 是第 i 个样本的状态和回报，不要求相邻样本来自同一条轨迹：

$$\mathcal{L}_{\text{MC-V}}(\phi) = \frac{1}{2N} \sum_{i=1}^N (V_\phi(s_i) - G_i)^2.$$

同一个状态在不同 rollout 中可能对应不同的 return。例如从某个状态出发，三次采样得到

$$G_t^{(1)} = 6, \quad G_t^{(2)} = 14, \quad G_t^{(3)} = 10.$$

这并不矛盾，因为未来动作和环境都可能是随机的。 $V^\pi(s_t)$ 要学习的不是其中某一个数，而是条件平均。若这些样本具有代表性，那么

$$\frac{6 + 14 + 10}{3} = 10$$

会逐渐逼近该状态的真实平均价值。

Critic 为什么能降低方差。 REINFORCE 直接把一次 realized return G_t 用作动作权重；Critic 则尝试把许多随机未来压缩成一个状态条件下的平均预测 $V_\phi(s)$ 。如果这个预测足够准确，它可以过滤掉一部分与当前动作无关的未来随机性，从而为 Actor 提供更稳定的参照。

但 Monte Carlo value fitting 仍然有两个明显限制。第一，必须等到足够长的未来甚至整个 episode 结束后才能得到 G_t ；第二， G_t 本身仍然包含大量未来随机性，

因此作为回归 target 方差较大。为了更早地更新 Critic，并进一步减少方差，需要引入 bootstrapping。

3.3 Bootstrapping、TD Error 与 Advantage 估计

为统一处理终止边界，令 $d_t = 1$ 表示这一步转移到 MDP 意义下的真正终止状态， $d_t = 0$ 表示之后仍有可继续估计的未来；若采样仅因外部 time limit 被截断而任务本身未终止，则应按 $d_t = 0$ 处理。对真正终止状态，理论上约定其后续价值为零；在函数近似实现中显式乘 $1 - d_t$ 可以避免依赖网络在 terminal 输入上的任意输出。

从完整未来拆出第一步。Reward-to-Go 满足递推关系

$$G_t = r_t + \gamma G_{t+1}.$$

把它代入 Q^π 的定义，并利用 MDP 的 Markov 结构，可以得到 Bellman 期望关系

$$Q^\pi(s_t, a_t) = \mathbb{E}[r_t + \gamma(1 - d_t)V^\pi(s_{t+1}) \mid s_t, a_t].$$

对 MDP 意义下的真正终止状态，约定 $V^\pi(s_{t+1}) = 0$ ；在函数近似实现中则显式使用终止 mask，避免依赖网络在 terminal 输入上的任意输出。这里最需要注意的是：**外层条件期望不能省略。**

数学基础提示： Q^π 与 V^π 的 Bellman 期望关系，以及其中 Markov 条件的严格使用，详见附录 A.3。

$r_t + \gamma(1 - d_t)V^\pi(s_{t+1})$ **不是一般意义下的精确 Q^π** 。固定当前 $s_t = s$ 、 $a_t = a$ 后，环境仍然可能随机地产生不同的 reward 和下一状态。于是

$$Y_t := r_t + \gamma(1 - d_t)V^\pi(s_{t+1}) \tag{3.3}$$

本身仍然是随机变量，而

$$Q^\pi(s, a) = \mathbb{E}[Y_t \mid s_t = s, a_t = a]$$

才是这些可能结果的条件平均。

例如，固定同一个 (s, a) 后，假设有

$$70\%: \quad r_t = 2, \quad s_{t+1} = s_A, \quad V^\pi(s_A) = 10,$$

$$30\%: \quad r_t = -1, \quad s_{t+1} = s_B, \quad V^\pi(s_B) = 4,$$

并取 $\gamma = 0.9$ 。那么两种可能的一步结果分别是

$$2 + 0.9 \times 10 = 11$$

和

$$-1 + 0.9 \times 4 = 2.6.$$

真正的动作价值是二者的平均：

$$Q^\pi(s, a) = 0.7 \times 11 + 0.3 \times 2.6 = 8.48.$$

因此一次实际 transition 得到的 11 或 2.6 都只是 $Q^\pi(s, a)$ 的一个单步样本，而不是一般情况下的精确动作价值。例如，当当前 reward 和下一状态在给定 (s, a) 后完全确定时，外层期望会退化成单个值；更一般地，只要 $r_t + \gamma(1 - d_t)V^\pi(s_{t+1})$ 在给定 (s, a) 后几乎处处为同一个值即可，reward 与下一状态各自仍可随机。

为什么 $V^\pi(s_{t+1})$ 能参与评价当前动作。 $V^\pi(s_{t+1})$ 本身确实只表示“已经到达 s_{t+1} 以后，按照策略继续行动的平均未来价值”。当前动作 a_t 的影响没有消失，而是已经通过

$$a_t \longrightarrow (r_t, s_{t+1})$$

进入了式 (3.3)。动作执行完后，从 s_{t+1} 开始的未来才重新由策略 π 接管，因此后半段自然由 $V^\pi(s_{t+1})$ 表示。

Bootstrapping. 真实的 $V^\pi(s_{t+1})$ 仍然未知，但 Critic 已经给出近似 $V_\phi(s_{t+1})$ 。沿用上面的终止标记 d_t ，可以用

$$y_t^{\text{TD}} = r_t + \gamma(1 - d_t)V_\phi(s_{t+1})$$

作为当前状态价值的单步 target，并让

$$V_\phi(s_t) \longrightarrow y_t^{\text{TD}}.$$

这种“用自己对下一状态的当前估计来帮助更新当前状态”的方式称为 **bootstrapping**。它不必等到整个未来完全发生以后再更新，因此能够更早地产生学习信号。

TD Error。当前价值预测与单步 target 的差定义为 Temporal-Difference error：

$$\hat{\delta}_t = r_t + \gamma(1 - d_t)V_\phi(s_{t+1}) - V_\phi(s_t).$$

从 Critic 的角度看，它表示“新的单步目标与原有价值预测相差多少”。若 $\hat{\delta}_t > 0$ ，说明当前预测偏低；若 $\hat{\delta}_t < 0$ ，说明当前预测偏高。

TD Error 为什么又能评价 Actor。如果暂时假设 Critic 是精确的，即 $V_\phi = V^\pi$ ，定义

$$\delta_t^\pi = r_t + \gamma(1 - d_t)V^\pi(s_{t+1}) - V^\pi(s_t),$$

那么有

$$\mathbb{E}[\delta_t^\pi \mid s_t, a_t] = A^\pi(s_t, a_t).$$

因此 δ_t^π 是 Advantage 的一个单步随机估计。实践中 Critic 只是近似值，所以通常写成

$$\hat{A}_t \approx \hat{\delta}_t = r_t + \gamma(1 - d_t)V_\phi(s_{t+1}) - V_\phi(s_t).$$

严格证明以及 Critic 误差如何进入该估计，放在附录 A.3。

数学基础提示：TD Error 作为 Advantage 单步估计的条件期望推导，详见附录 A.3。

3.4 Actor-Critic 算法与实现

Actor-Critic 把前面的两部分学习组合在同一个循环中：Actor 产生数据并改善策略，Critic 评价当前策略并为 Actor 提供 Advantage 估计。有两种实现方法：

Batch Actor-Critic。一种直接的 batch 形式与前两章的训练方式最接近：

1. 使用当前策略 π_θ 收集一批 trajectory；
2. 用这些 trajectory 的 sampled reward sums 拟合 $V_\phi(s)$ ；

3. 对每个 transition 计算一步 Advantage 估计

$$\hat{A}_t = r_t + \gamma(1 - d_t)V_\phi(s_{t+1}) - V_\phi(s_t);$$

4. 用

$$\hat{\mathbf{g}} = \frac{1}{N} \sum_{i=1}^N \hat{A}_i \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_i | s_i)$$

更新 Actor;

5. 用更新后的策略重新采样, 再重复上述过程。

上式按有效 transition 平均。对同一批 m 条完整轨迹、使用相同样本权重时, 记按轨迹求和再平均的方向为 $\hat{\mathbf{g}}_{\text{episode}}$, 按 transition 平均的方向为 $\hat{\mathbf{g}}_{\text{step}}$, 则

$$\hat{\mathbf{g}}_{\text{step}} = \frac{m}{N} \hat{\mathbf{g}}_{\text{episode}}, \quad N = \sum_{j=1}^m T_j.$$

对固定 batch, 这只是正的尺度变化; 若 episode 长度随机, N 也随机, 不能因此宣称 transition 平均仍是第二章 episodic 目标梯度的无偏估计。本章将其作为经验 Actor 更新方向。

这种形式与 REINFORCE 的区别不是 “Policy Gradient 消失了”, 而是用于加权 log probability 的量从单次 Monte Carlo return, 变成了由 Critic 帮助构造的 Advantage 估计。

Online Actor-Critic. 如果 Critic 使用 bootstrapped target, 那么每得到一个 transition $(s_t, a_t, r_t, s_{t+1}, d_t)$ 就可以构造一次局部更新。

1. Actor 根据 $\pi_{\boldsymbol{\theta}}(\cdot | s_t)$ 采样动作 a_t ;
2. 环境返回奖励 r_t 、下一个状态 s_{t+1} 与真正终止标记 d_t ;
3. Critic 计算当前状态和下一状态的价值估计, 并构造 TD target

$$y_t^{\text{TD}} = r_t + \gamma(1 - d_t)V_\phi(s_{t+1});$$

4. 由此得到 TD error

$$\hat{A}_t = y_t^{\text{TD}} - V_\phi(s_t) = r_t + \gamma(1 - d_t)V_\phi(s_{t+1}) - V_\phi(s_t);$$

5. Critic 使 $V_\phi(s_t)$ 接近 y_t^{TD} , Actor 则使用 $\hat{A}_t$ 更新策略。

这正体现了 bootstrapping 的优势：不必等完整 episode 结束，当前 transition 本身已经能够同时训练 Critic 和 Actor。

两个 loss。 在深度学习实现中，可以把两个更新写成两个损失。标准一步 Actor-Critic 通常采用 TD 的 semi-gradient：bootstrap target 在本次 Critic 反向传播中视为常数。记 $\text{stopgrad}(\cdot)$ 表示停止梯度，则

$$\mathcal{L}_{\text{critic}}(\phi) = \frac{1}{2N} \sum_{i=1}^N [V_{\phi}(s_i) - (r_i + \gamma(1 - d_i) \text{stopgrad}(V_{\phi}(s'_i)))]^2.$$

若直接让梯度穿过 target 中的 $V_{\phi}(s'_i)$ ，优化的将是另一类 residual-gradient 目标，而不是这里所讲的标准 TD semi-gradient。Actor 反向传播时把 $\hat{A}_i$ 视为固定权重，即使 Actor 与 Critic 共享部分参数，也不从优势计算分支传播梯度。Actor 的 loss 为

$$\mathcal{L}_{\text{actor}}(\theta) = -\frac{1}{N} \sum_{i=1}^N \text{stopgrad}(\hat{A}_i) \log \pi_{\theta}(a_i | s_i).$$

与第二章一样，负号来自“深度学习框架默认最小化 loss，而策略目标希望做梯度上升”。

3.5 Actor-Critic 的统计特性与局限

Actor-Critic 并不是把 REINFORCE 完全替换成了另一套优化原理。Actor 仍然使用 Policy Gradient；真正改变的是如何估计策略梯度中的回报或 Advantage 权重。

Monte Carlo 与 Bootstrapping。 REINFORCE 的 Reward-to-Go

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

真实地使用了后续采样 reward，因此不依赖价值函数预测，但会继承整段未来的随机性。一步 Actor-Critic 使用 $r_t + \gamma(1 - d_t)V_{\phi}(s_{t+1})$ 把长未来压缩为 Critic 的一个状态价值预测，通常能够显著减少方差，但代价是开始依赖 V_{ϕ} 的准确性。

Critic 可以有两种不同用法。 第一种用法只是把 Critic 当作状态相关 baseline，仍然保留 Monte Carlo return：

$$\hat{A}_t^{\text{MC}} = G_t - V_{\phi}(s_t).$$

只要这个 **baseline** 在给定动作采样前历史后不依赖当前采样动作，减去它不会改变相应加权 **score** 的期望；如果它能够较好预测 G_t ，通常会降低方差。这里的独立性是统计条件，不只是网络输入不含动作：若用同一 **batch** 的动作和回报拟合 **baseline** 后再回用，不能仅凭输入为状态就宣称无偏；停止梯度也不等于统计独立。

第二种用法是本章重点讨论的一步 **bootstrapped Actor-Critic**：

$$\hat{A}_t^{[1]} = r_t + \gamma(1 - d_t)V_\phi(s_{t+1}) - V_\phi(s_t).$$

这种估计进一步用 **Critic** 替代了大部分真实未来 **reward**，因此方差通常更低；但如果 **Critic** 不完美，估计本身可能带来偏差。两者的区别不能混为一谈。

Bias-Variance Tradeoff。可以把两种极端理解为：

更多真实未来 **reward** $\iff$ 更少依赖 **Critic**，通常方差更大，

而

更早使用 **bootstrap** $\iff$ 更依赖 **Critic**，通常方差更小但更受近似误差影响。

Actor-Critic 的很多后续改进，本质上都围绕这一 **bias-variance tradeoff** 展开。

Actor-Critic 的核心不是“把 **Policy Gradient** 换掉”，而是让 **Critic** 学习当前策略的价值函数，并用价值估计构造比原始 **Monte Carlo return** 更有参照、更低方差的 **Advantage** 信号。一步 **TD error** $r_t + \gamma(1 - d_t)V_\phi(s_{t+1}) - V_\phi(s_t)$ 同时连接了 **Critic** 的价值学习与 **Actor** 的策略更新。

PPO：近端策略优化

第三章已经给出了 Actor-Critic 的基本更新形式：利用 Advantage 判断当前动作相对于状态基线究竟更好还是更差，再用

$$\hat{A}_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

更新策略。

但这里还留下一个问题：即使更新方向是合理的，一次参数更新仍然可能使策略发生过大的变化。尤其当一批 rollout 被重复用于若干次优化时，当前策略会逐渐偏离真正产生这些数据的旧策略。此时，原来的训练信号虽然仍然指出“应该提高还是降低某个动作的概率”，却没有直接回答“已经改变了多少”。

Proximal Policy Optimization (PPO) 的核心，就是在 Actor-Critic 的基础上显式比较新旧策略，并限制一次更新中继续把策略推得更远所能获得的收益。实际训练中，PPO 又常与 Generalized Advantage Estimation (GAE) 配合，用于构造较稳定的 Advantage 估计。

本章沿着

新旧策略的概率比 $\rightarrow$ 裁剪目标 $\rightarrow$ GAE $\rightarrow$ PPO 算法

这条主线展开。

4.1 新旧策略的概率比

设当前有一批由旧策略 π_{old} 采样得到的数据。开始优化后，参数不断更新，对应的当前策略记为 π_{θ} 。对于同一个已经采样到的状态-动作对 (s_t, a_t) ，最直接的问题是：当前策略相对于旧策略，究竟把这个动作的概率改变了多少？

定义新旧策略的概率比

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}.$$

PPO 原论文常把这个概率比记作 $r_t(\theta)$ ；本书已经用 r_t 表示即时 reward，因此这里使用 $\rho_t(\theta)$ 。

这个量的含义非常直接：

$\rho_t = 1 \implies$ 新旧策略对动作 a_t 的概率相同，

$\rho_t > 1 \implies$ 当前策略提高了动作 a_t 的概率，

$\rho_t < 1 \implies$ 当前策略降低了动作 a_t 的概率。

因此， ρ_t 把“参数发生了多大变化”转化成了一个更直接的问题：**已采样动作的概率发生了多大相对变化**。

这比直接比较参数 θ 更有意义。神经网络参数空间中的相同步长，并不意味着策略分布发生同样大小的变化；而 probability ratio 直接作用在策略概率本身。

从 Actor-Critic 到 surrogate objective。在第三章中，如果 $\hat{A}_t > 0$ ，说明动作 a_t 相对于当前状态的基线表现较好，应提高其概率；如果 $\hat{A}_t < 0$ ，则应降低其概率。

利用概率比，可以把这种方向关系写成

$$L^{\text{sur}}(\theta) = \mathbb{E}_t \left[\rho_t(\theta) \hat{A}_t \right]. \quad (4.1)$$

这里 $\mathbb{E}_t[\cdot]$ 表示对当前 rollout batch 中有效时间步的经验平均，而不是另一个未声明的理论状态分布期望。 $\hat{A}_t$ 由旧策略采样得到，在当前一轮策略优化中被视为固定的训练信号。

若 $\hat{A}_t > 0$ ，增大 ρ_t 会提高目标；若 $\hat{A}_t < 0$ ，减小 ρ_t 会提高目标。这个方向与 Advantage 的含义一致。

从 Importance Sampling 的角度看， ρ_t 还可以理解为对“数据来自旧策略，而当前优化的是新策略”这一差异的局部修正。从旧策略期望到 surrogate objective 的严格推导见附录 A.4.2。

但式 (4.1) 还存在一个明显问题：只要方向正确，目标就会持续鼓励 ρ_t 继续增大或减小。也就是说，它告诉我们“往哪边改”，却没有给出“什么时候应该停止继续推远”的机制。

这正是 PPO-Clip 要解决的问题。

4.2 裁剪目标与近端策略更新

先考虑一个 Advantage 为正的样本。若 $\hat{A}_t > 0$ ，提高动作 a_t 的概率是合理的，因此希望 $\rho_t > 1$ 。但如果 ρ_t 已经从 1 变成 1.2、1.5，甚至更大，继续提高这个动作的概率是否仍然应该获得同样的优化收益？

对于负 Advantage 也有同样的问题。若 $\hat{A}_t < 0$ ，降低动作概率是合理的，但把一个动作的概率持续压低，也可能使新策略迅速远离采样数据的旧策略。

PPO 的思想不是禁止策略改变，而是让更新保持在旧策略附近。为此引入裁剪算子

$$\text{clip}(\rho_t(\boldsymbol{\theta}), 1 - \epsilon, 1 + \epsilon),$$

其中 $\epsilon > 0$ 控制允许的相对变化范围。

PPO-Clip 的目标写成

$$L^{\text{CLIP}}(\boldsymbol{\theta}) = \mathbb{E}_t \left[\min \left(\rho_t(\boldsymbol{\theta}) \hat{A}_t, \text{clip}(\rho_t(\boldsymbol{\theta}), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]. \quad (4.2)$$

理解这个目标，关键是分别看 $\hat{A}_t$ 的正负。

当 $\hat{A}_t > 0$ 。当前动作比基线好，因此应提高它的概率。只要 ρ_t 从 1 向上移动，原始 surrogate objective 就会增加。

但当

$$\rho_t > 1 + \epsilon$$

以后，裁剪项不再继续增大。于是继续把这个好动作的概率推得更高，不再获得额外的 surrogate objective 收益。

当 $\hat{A}_t < 0$ 。当前动作比基线差，因此应降低它的概率。此时减小 ρ_t 会提高原始目标。

但当

$$\rho_t < 1 - \epsilon$$

以后，裁剪项阻止“继续把这个坏动作的概率压得更低”获得额外收益。

因此，PPO-Clip 的核心并不是把 probability ratio 强制限制在某个区间，而是：

当策略已经朝正确方向改变得足够多时，继续沿同一方向把策略推得更远，不再得到额外优化收益。

为什么还要取 min。 裁剪并不是为了掩盖错误方向的更新。如果策略朝错误方向移动，未裁剪项可能给出更差的目标值。取两者中的较小值，会保留这个更差的结果，而不会因为 clip 把它“修饰”成一个更好的边界值。

因此可以把 PPO-Clip 的逻辑概括为：

方向正确但走得过远 $\implies$ 停止继续奖励，

而

方向错误 $\implies$ 仍然保留惩罚。

正负 Advantage 下的完整分段形式见附录 A.4.3。

Clip 不是硬约束。 需要特别注意，式 (4.2) 裁剪的是目标函数中的收益，而不是强制要求

$$1 - \epsilon \leq \rho_t(\theta) \leq 1 + \epsilon$$

始终成立。一次参数更新后，某些样本的 ratio 完全可能落在裁剪区间之外。

PPO 所做的是改变这些样本对优化目标的贡献，从而减少“继续远离旧策略”的激励。它是一种近端更新思想，而不是严格的参数投影或 trust-region 硬约束。

到这里，PPO 已经回答了“怎样限制策略更新”的问题。但式 (4.2) 仍然依赖一个重要量 $\hat{A}_t$ 。PPO 本身并不规定 Advantage 必须怎样计算；在实际使用中，最常见的选择之一就是 GAE。

4.3 广义优势估计

第三章已经给出一步 TD error。沿用其中的终止标记，令 $d_t = 1$ 表示这一步转移到 MDP 意义下的真正终止状态，否则 $d_t = 0$ 。于是

$$\hat{\delta}_t = r_t + \gamma(1 - d_t)V_\phi(s_{t+1}) - V_\phi(s_t).$$

它可以作为 Advantage 的一步估计：只观察当前 reward，然后立即使用 Critic 对未来做 bootstrap。若采样仅因外部 time limit 被截断而任务本身未终止，则仍应按 $d_t = 0$ 处理。

这种估计的优点是只依赖很短的随机未来，方差通常较低；缺点是较早依赖 Critic，如果价值函数还不准确，估计会受到较明显的近似误差影响。

另一端是更长步数的 return。下面的 n -step 公式用于同一条连续轨迹的 n 步片段，不跨 reset，且片段内部不含真正终止；末步允许恰好到达终止状态，此时用 $1 - d_{t+n-1}$ 取消端点 bootstrap。方括号上标 $[n]$ 表示步数，区别于样本序号 (i) 。定义

$$G_t^{[n]} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n (1 - d_{t+n-1}) V_\phi(s_{t+n}),$$

并令

$$\hat{A}_t^{[n]} = G_t^{[n]} - V_\phi(s_t).$$

随着 n 增大，估计会使用更多真实 reward，减少对短期 bootstrap 的依赖，但同时也会把更多未来动作、环境转移和 reward 的随机性带进来。

因此，Advantage 估计面临和前面 Monte Carlo / TD 类似的取舍：

短时间尺度 $\iff$ 更多依赖 Critic，通常方差较低，

长时间尺度 $\iff$ 更多依赖真实轨迹，通常方差较高。

从 n -step 到 GAE。 利用 telescoping cancellation，可以把 n -step Advantage 写成连续 TD error 的累积：

$$\hat{A}_t^{[n]} = \hat{\delta}_t + \gamma \hat{\delta}_{t+1} + \cdots + \gamma^{n-1} \hat{\delta}_{t+n-1}.$$

这提示我们，不一定必须固定某个 n 。可以让不同时间尺度的 TD error 同时参与，只是越远的项权重越小。

Generalized Advantage Estimation (GAE) 正是这样做：

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \hat{\delta}_t + (\gamma \lambda) \hat{\delta}_{t+1} + (\gamma \lambda)^2 \hat{\delta}_{t+2} + \cdots,$$

其中 $\lambda \in [0, 1]$ 控制较远 TD error 的衰减速度。上式先表示同一连续轨迹上的级数；无限情形需要可和条件，有限 rollout 的截断与递推边界按下文处理。

当 $\lambda = 0$ 时，只有当前一步 TD error 被保留：

$$\hat{A}_t^{\text{GAE}} = \hat{\delta}_t.$$

随着 λ 增大，后续 TD error 衰减得更慢，当前 Advantage 会吸收更长时间范围的信息。在到达真正终止状态的完整 episode 中， $\lambda = 1$ 给出 Monte Carlo return 减基线；若 rollout 仅在非终止状态截断，则仍保留端点 bootstrap，不能称为完整 Monte Carlo return。

这里需要区分 γ 与 λ ：按照本书的 convention，第一章主策略目标保持未折扣 episodic 形式；这里的 γ 用于 discounted value / GAE，决定较远未来 reward 在估计器中的权重，并形成相对于主目标的 bias-variance tradeoff。 λ 则只用于 Advantage estimator，控制不同时间尺度 TD residual 的组合。

对于有限 rollout，必须区分价值 bootstrap 和优势递推的边界。记 V_{old} 为采样时冻结的 Critic， d_t 仍只表示真正终止；另令 $c_t = 1$ 表示下一条 TD residual 属于同一条连续轨迹且仍在当前可用片段内。在真正终止、reset 或片段末尾，均取 $c_t = 0$ 。于是

$$\hat{\delta}_t = r_t + \gamma(1 - d_t)V_{\text{old}}(s_{t+1}) - V_{\text{old}}(s_t),$$

以及

$$\hat{A}_t = \hat{\delta}_t + \gamma\lambda c_t \hat{A}_{t+1}. \quad (4.3)$$

若当前可用片段的 transition 编号为 $0, \dots, K - 1$ ，可初始化 $\hat{A}_K = 0$ ，再从 $K - 1$ 向前递推。真正终止处不 bootstrap，也不连接后续优势；外部 time-limit 截断处保留 $d_t = 0$ 以 bootstrap，但因片段结束或 reset，仍令 $c_t = 0$ 。此时 s_{t+1} 必须是截断前实际获得的最后状态，不能替换成 reset 后的新初始状态。batch 中每个环境的轨迹分别按这些边界计算，不能把不同 episode 或不同环境串接起来。

GAE 与加权 n -step Advantage 的严格等价关系见附录 A.4.1。

GAE 并不是 PPO 的定义本身。它解决的是 Advantage estimator 的问题：一步 TD 太早依赖 Critic，完整 Monte Carlo 又容易具有较大方差；GAE 用 λ 把不同时间尺度的信息组合起来，在两者之间取得折中。

4.4 PPO 算法

前面三节分别回答了三个问题：Probability Ratio 描述新旧策略发生了多大相对变化，PPO-Clip 限制沿有利方向继续把策略推得过远，而 GAE 为策略更新提供较稳定的 Advantage 估计。把它们放回 Actor-Critic 框架，就得到 PPO 的基本算法结构。

第一步：用旧策略采样。 固定当前策略为 π_{old} ，用它与环境交互得到一批 rollout：

$$(s_t, a_t, r_t, s_{t+1}, d_t, c_t).$$

同时保留旧策略对已采样动作的概率信息，以及 rollout 时 Critic 的冻结价值估计，记为 $V_{\text{old}}(s_t)$ 。在随后针对这批数据执行的多个优化 epoch 中， π_{old} 、 V_{old} 、 $\hat{A}_t$ 与由它们构造的 value target 都作为固定数据，不从这些 target 向当前参数反向传播。

这里的关键是：这一批数据由 π_{old} 产生，因此在后面的策略优化中， π_{old} 必须始终作为固定参照。

第二步：估计 Advantage。 利用冻结的 V_{old} 计算 TD residual，并按式 (4.3) 的片段边界由 GAE 得到

$$\hat{A}_t.$$

这一步回答“动作 a_t 相对于当前状态的基线究竟表现得更好还是更差”。

在常见的 PPO + GAE 组合中，还可以构造与之对应的 value target

$$\hat{R}_t = \hat{A}_t + V_{\text{old}}(s_t),$$

供 Critic 回归使用。

第三步：更新策略。 当前策略 π_{θ} 对同一批状态-动作重新计算动作概率，并构造

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}.$$

随后最大化 clipped objective

$$L^{\text{CLIP}}(\theta).$$

这里可以看清三个量的分工：

$\hat{A}_t$ 决定动作应当被提高还是降低概率，

$\rho_t(\theta)$ 描述当前策略相对于旧策略已经改变了多少,

clip 限制继续把策略推得更远所能获得的收益.

Critic 则继续拟合 value target, 例如

$$\mathcal{L}_V(\phi) = \frac{1}{2} \mathbb{E}_t \left[(V_\phi(s_t) - \hat{R}_t)^2 \right].$$

第四步：重新采样。 当这一批旧策略数据已经完成预定的策略更新后，更新后的当前策略成为下一轮新的采样策略，再重新与环境交互。

因此，PPO 的核心循环可以概括为

$$\pi_{\text{old}} \xrightarrow{\text{采样}} \text{rollout} \xrightarrow{\text{GAE}} \hat{A}_t \xrightarrow{\text{ratio} + \text{clip}} \pi_\theta \xrightarrow{\text{重新采样}} \dots$$

PPO 的核心并不是提出一种新的价值函数，也不是要求必须使用某一种 Advantage estimator。它真正新增的机制，是用新旧策略的 probability ratio 描述策略变化，并通过 clipped surrogate objective 抑制一次更新中过度远离旧策略的激励。GAE 则是与 PPO 非常常见的配套方法，用来提供较稳定的 Advantage 估计。

PPO 可以看作 Actor-Critic 上的一次近端策略更新：Advantage 决定往哪个方向改变动作概率，probability ratio 衡量已经改变了多少，clip 决定什么时候不再奖励继续推远。

附录

A.1 Policy Gradient 的严格推导

正文给出了从期望回报到 Policy Gradient 的主推导。本附录不重复基础概率与微积分定义，而是补充几个在正文中容易被压缩、却直接关系到公式是否成立的步骤：轨迹联合概率如何分解，log-derivative trick 如何把轨迹概率梯度改写成 score function，以及为什么环境动力学最终不会出现在策略参数的解析梯度中。

A.1.1 轨迹联合概率的严格分解

考虑有限时域轨迹

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_{T-1}, a_{T-1}, r_{T-1}, s_T).$$

在不使用任何条件独立性之前，联合概率可以按概率链式法则逐项展开。以两步轨迹

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, s_2)$$

为例，有

$$\begin{aligned} P_{\theta}(\tau) &= P(s_0)P_{\theta}(a_0 | s_0)P(s_1, r_0 | s_0, a_0) \\ &\quad \times P_{\theta}(a_1 | s_0, a_0, r_0, s_1)P(s_2, r_1 | s_0, a_0, r_0, s_1, a_1). \end{aligned}$$

若采用标准 Markov 策略，则动作只依赖当前状态：

$$P_{\theta}(a_t | s_0, a_0, r_0, \dots, s_t) = \pi_{\theta}(a_t | s_t).$$

若环境满足 Markov 性，则下一状态与即时奖励的联合分布只依赖当前状态和动作：

$$P(s_{t+1}, r_t | s_0, a_0, r_0, \dots, s_t, a_t) = P(s_{t+1}, r_t | s_t, a_t).$$

因此一般的有限轨迹可分解为

$$P_{\theta}(\tau) = \mu_0(s_0) \prod_{t=0}^{T-1} \pi_{\theta}(a_t | s_t) P(s_{t+1}, r_t | s_t, a_t). \quad (\text{A.1})$$

这里的下标 θ 并不表示环境也由 θ 参数化。它只表示：式 (A.1) 中含有策略因子 $\pi_{\theta}(a_t | s_t)$ ，因此改变策略参数会改变整条轨迹的概率。也正因为如此，文献中的 $\tau \sim \pi_{\theta}$ 应理解为“在固定环境中使用 π_{θ} rollout 轨迹”的简写；严格地说，随机轨迹服从的是 $P_{\theta}(\tau)$ 。

A.1.2 从期望回报到 likelihood-ratio 形式

以下推导假定环境与初始分布不含待优化的策略参数，策略及轨迹密度在共同参考测度下可微，并在所考虑的参数邻域具有不随参数移动的支持集。对数与 score 仅在正密度支持上使用；还假定相关回报与 score 乘积可积，且梯度可以与求和或积分交换。允许可变 episode 长度时，轨迹空间包含所有可能长度，并要求相应求和和求导交换仍成立。这些条件也用于后续 score 零均值及 baseline 推导；若支持集随参数移动，则必须另行检查边界项，不能直接套用结论。

设策略目标为

$$J(\theta) = \sum_{\tau} P_{\theta}(\tau) R(\tau), \quad (\text{A.2})$$

其中一旦轨迹 τ 给定， $R(\tau)$ 由任务和环境决定，不再显式依赖策略参数。对式 (A.2) 求梯度：

$$\nabla_{\theta} J(\theta) = \sum_{\tau} R(\tau) \nabla_{\theta} P_{\theta}(\tau). \quad (\text{A.3})$$

对于任意正的可微概率项 $P_{\theta}(\tau)$ ，有

$$\nabla_{\theta} P_{\theta}(\tau) = P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau). \quad (\text{A.4})$$

将式 (A.4) 代入式 (A.3)：

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \sum_{\tau} P_{\theta}(\tau) R(\tau) \nabla_{\theta} \log P_{\theta}(\tau) \\ &= \mathbb{E}_{\tau \sim P_{\theta}} [R(\tau) \nabla_{\theta} \log P_{\theta}(\tau)]. \end{aligned} \quad (\text{A.5})$$

这一步的关键作用不是“引入一个对数”本身，而是把概率分布对参数的导数，变成了一个可以直接从采样轨迹上计算的 log-likelihood 梯度。后续算法只需要知道策略对已采样动作赋予的 log probability，而不需要枚举所有可能轨迹。

A.1.3 环境动力学为什么在梯度中消失

由式 (A.1),

$$\begin{aligned} \log P_{\theta}(\tau) = & \log \mu_0(s_0) + \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t | s_t) \\ & + \sum_{t=0}^{T-1} \log P(s_{t+1}, r_t | s_t, a_t). \end{aligned}$$

对策略参数 θ 求梯度。标准 policy optimization 假定初始状态分布与环境动力学不含策略参数，因此

$$\nabla_{\theta} \log \mu_0(s_0) = 0,$$

以及

$$\nabla_{\theta} \log P(s_{t+1}, r_t | s_t, a_t) = 0.$$

于是

$$\nabla_{\theta} \log P_{\theta}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t). \quad (\text{A.6})$$

这并不意味着环境“没有作用”。环境仍然决定轨迹中实际访问到哪些状态、获得哪些奖励，从而改变采样分布与 $R(\tau)$ ；消失的只是“环境转移概率对策略参数的直接导数”。因此，Policy Gradient 可以用于不可微模拟器、真实物理系统等场景：只要能够与环境交互并得到 reward，就不要求环境本身对 θ 可微。

如果环境转移模型本身也显式共享待优化参数 θ ，那么上面的零梯度条件不再成立，需要额外保留环境项。本书讨论的是标准的 model-free policy optimization，因此采用环境与策略参数分离的设定。

A.1.4 完整的轨迹形式 Policy Gradient

把式 (A.6) 代入式 (A.5)，得到

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim P_{\theta}} \left[R(\tau) \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right].$$

等价地，也可以把 $R(\tau)$ 放入求和：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim P_{\theta}} \left[\sum_{t=0}^{T-1} R(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]. \quad (\text{A.7})$$

这两个式子与正文中的主公式完全等价。它们强调的是 trajectory likelihood 的视角：先对整条轨迹的概率求 log-gradient，再利用乘积结构把梯度拆成每个时间步的 policy score。

A.1.5 常见简写与严格写法的对应关系

强化学习资料中经常出现

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau)]. \quad (\text{A.8})$$

如果没有进一步定义期望中的随机变量，式 (A.8) 只是强调单时间步结构的教学性简写。对应到轨迹 likelihood 推导，更完整的形式是式 (A.7)，其中对时间步 t 的求和是显式的。

另一类常见表达使用状态价值来组织策略梯度。为保留本书未折扣 episodic 目标的尺度，可以显式保留时间求和：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim P_{\theta}} \left[\sum_{t=0}^{T-1} Q^{\pi_{\theta}}(s_t, a_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right].$$

这里 $Q^{\pi_{\theta}}$ 采用未折扣 reward-to-go；固定有限时域时，状态包含剩余时间。若进一步把时间步吸收到归一化的状态访问分布中，则必须同时保留归一化尺度：例如固定长度 T 下，均匀选一个时间步所得的状态分布对应时间平均，而上式是时间求和，两者相差因子 T 。可变 episode 长度还涉及长度加权，不能省略采样与归一化约定。对于理解 REINFORCE 以及后续 LLM 中的 log-probability 更新，轨迹形式更直接，因此正文采用这一写法。

A.2 REINFORCE 中的若干关键推导

REINFORCE 的算法形式很短, 但其中有几个结论并不显然: 为什么有限条 trajectory 的平均可以估计理论策略梯度, 为什么 Reward-to-Go 可以替代整条轨迹回报, 以及为什么这种替换不会改变期望梯度。本附录集中推导这些与第二章直接相关、但放在正文中会打断算法叙事的细节。

A.2.1 Monte Carlo 策略梯度估计的无偏性

把一条随机轨迹对策略梯度的贡献记为

$$\mathbf{g}(\tau) = R(\tau) \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

根据 Policy Gradient 的轨迹形式,

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim P_{\theta}}[\mathbf{g}(\tau)].$$

若使用当前策略独立采样 m 条轨迹

$$\tau^{(1)}, \dots, \tau^{(m)} \sim P_{\theta},$$

定义 Monte Carlo 估计器

$$\hat{\mathbf{g}} = \frac{1}{m} \sum_{i=1}^m \mathbf{g}(\tau^{(i)}).$$

则由期望的线性性,

$$\begin{aligned} \mathbb{E}[\hat{\mathbf{g}}] &= \frac{1}{m} \sum_{i=1}^m \mathbb{E}[\mathbf{g}(\tau^{(i)})] \\ &= \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} J(\theta) \\ &= \nabla_{\theta} J(\theta). \end{aligned}$$

因此, 在采样确实来自当前策略且相关正则条件成立时, REINFORCE 的 batch 平均是理论策略梯度的无偏估计。

这里的“无偏”只说明重复采样后的平均值正确，并不意味着单个 batch 的 $\hat{g}$ 接近真实梯度。单次估计仍可能有很大方差，这也是 REINFORCE 训练不稳定的主要来源之一。

A.2.2 Score Function 的零均值性质

Reward-to-Go 推导依赖一个关键恒等式。对固定状态 s ，动作由 $a \sim \pi_{\theta}(\cdot | s)$ 采样，则

$$\begin{aligned}
 \mathbb{E}_{a \sim \pi_{\theta}(\cdot | s)} [\nabla_{\theta} \log \pi_{\theta}(a | s)] &= \sum_a \pi_{\theta}(a | s) \nabla_{\theta} \log \pi_{\theta}(a | s) \\
 &= \sum_a \nabla_{\theta} \pi_{\theta}(a | s) \\
 &= \nabla_{\theta} \sum_a \pi_{\theta}(a | s) \\
 &= \nabla_{\theta} 1 \\
 &= 0.
 \end{aligned} \tag{A.9}$$

连续动作情形将求和换成积分；在附录 A.1 声明的共同支持集、可积性与梯度交换条件下，结论相同。若支持集随参数移动，必须先核对边界项，不能仅把求和符号换成积分。

因此，只要某个量 b 在给定状态和历史后不依赖当前采样动作 a_t ，就有

$$\mathbb{E} [b \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] = 0. \tag{A.10}$$

后续 baseline 方法，以及 Reward-to-Go 去掉过去 reward，都利用了这一结构。

A.2.3 为什么过去的 Reward 可以被去掉

考虑未折扣的 episodic return：

$$R(\tau) = \sum_{k=0}^{T-1} r_k.$$

第 t 个时间步在策略梯度中的贡献为

$$\mathbb{E} [R(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)].$$

把总回报拆成过去和未来：

$$R(\tau) = \underbrace{\sum_{k=0}^{t-1} r_k}_{B_t} + \underbrace{\sum_{k=t}^{T-1} r_k}_{G_t}.$$

于是

$$\begin{aligned} & \mathbb{E}[R(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] \\ &= \mathbb{E}[B_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] + \mathbb{E}[G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]. \end{aligned} \quad (\text{A.11})$$

关键在第一项。将动作采样前已经观察到的状态、动作和奖励全部纳入历史，记为

$$H_t = (s_0, a_0, r_0, \dots, s_{t-1}, a_{t-1}, r_{t-1}, s_t), \quad t \geq 1,$$

并令 $H_0 = (s_0)$ 、 $B_0 = 0$ 。于是 B_t 对 H_t 可测；给定该历史后，它已确定，不依赖接下来从 $\pi_{\theta}(\cdot | s_t)$ 采样出的 a_t 。使用迭代期望：

$$\begin{aligned} & \mathbb{E}[B_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] \\ &= \mathbb{E}_{H_t} [B_t \mathbb{E}_{a_t \sim \pi_{\theta}(\cdot | s_t)} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) | H_t]] \\ &= 0, \end{aligned}$$

其中最后一步使用式 (A.9)。

因此式 (A.11) 中只剩未来回报：

$$\mathbb{E}[R(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] = \mathbb{E}[G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)].$$

这说明这里的未折扣 **Reward-to-Go** 并不是一个仅凭直觉加入的启发式，而是在期望意义下与第一章的未折扣策略梯度严格等价；它只是去掉了当前动作不可能影响的过去 reward，因此通常能减少无关噪声。若进一步令 $\gamma < 1$ 并改用 **discounted reward-to-go**，则要按下一小节区分“折扣目标的严格梯度”和“相对于本书未折扣主目标的 discounted approximation”。

A.2.4 折扣 Reward-to-Go 的记号细节

如果目标从时间零开始定义为

$$J(\theta) = \mathbb{E} \left[\sum_{k=0}^{T-1} \gamma^k r_k \right], \quad (\text{A.12})$$

那么第 t 步去掉过去 reward 后保留下来的量是

$$\sum_{k=t}^{T-1} \gamma^k r_k = \gamma^t \underbrace{\sum_{k=t}^{T-1} \gamma^{k-t} r_k}_{G_t}.$$

因此，若严格对应式 (A.12)，时间步 t 的权重应写成 $\gamma^t G_t$ 。

不少教材和代码直接使用 G_t ，常见原因包括：采用折扣状态访问分布来重新组织期望，或直接把 G_t 当作实践中的估计器权重。就本书而言，第一章主目标仍是未折扣 episodic return；因此 $\gamma < 1$ 时直接使用 discounted G_t 被明确视为算法性的 discounted approximation / bias-variance tradeoff，而不是把它写成与原始 $\nabla_{\theta} J(\theta)$ 完全相同的无偏估计。阅读不同资料时，需要先确认其目标函数和 visitation distribution 的定义，再比较公式。

A.2.5 多轨迹平均怎样影响梯度方差

设单条轨迹梯度贡献 $\mathbf{g}(\tau)$ 的协方差矩阵为

$$\Sigma = \text{Cov}[\mathbf{g}(\tau)].$$

若 m 条轨迹独立同分布，则

$$\text{Cov}[\hat{\mathbf{g}}] = \frac{1}{m} \Sigma.$$

因此增加 trajectory batch size 会降低 Monte Carlo 平均的方差；标准差尺度大约按 $1/\sqrt{m}$ 缩小。代价是每次更新前需要更多环境交互。

如果轨迹之间存在相关性，则

$$\text{Cov}[\hat{\mathbf{g}}] = \frac{1}{m^2} \left(\sum_i \text{Cov}[\mathbf{g}_i] + \sum_{i \neq j} \text{Cov}[\mathbf{g}_i, \mathbf{g}_j] \right),$$

额外的协方差项意味着方差未必严格按 $1/m$ 缩小。实际并行 rollout、连续任务或共享随机源时，这一点尤其值得注意。

更大的 batch 只能通过平均降低采样噪声，并不能消除 REINFORCE 中由于 return 本身尺度大、长时域 credit assignment 粗糙等结构性方差。这也是后续 baseline、advantage 与 Actor-Critic 方法仍然必要的原因。

A.3 Actor-Critic 中的关键推导

第三章把 REINFORCE 的 Monte Carlo return 进一步改写为价值函数与 Advantage，并使用一步 TD error 构造 Actor 的更新信号。本附录只展开其中三个与正文直接相关、但完整写在正文中会打断叙事的推导：从 Reward-to-Go 到 Q^π 与 Advantage、 Q^π 与 V^π 的 Bellman 期望关系，以及 TD Error 为什么可以作为 Advantage 的单步估计。

A.3.1 从 Reward-to-Go 到动作价值与 Advantage 形式

沿用第二章的记号，本附录先固定一个给定的 reward-to-go G_t （第三、四章默认使用 discounted G_t^γ 的简写）。下面从 G_t 到 Q^π 再到 A^π 的条件期望变换，对这个给定的 G_t 本身是严格的。第 t 个时间步的加权 score 项为

$$\mathbb{E}[G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)].$$

记

$$\mathbf{h}_t = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

由于 $\mathbf{h}_t$ 在给定 (s_t, a_t) 后已经确定，利用条件期望的塔式法则，

$$\begin{aligned} \mathbb{E}[G_t \mathbf{h}_t] &= \mathbb{E}[\mathbb{E}[G_t \mathbf{h}_t | s_t, a_t]] \\ &= \mathbb{E}[\mathbf{h}_t \mathbb{E}[G_t | s_t, a_t]]. \end{aligned}$$

根据动作价值函数定义

$$Q^{\pi}(s_t, a_t) = \mathbb{E}[G_t | s_t, a_t],$$

因此

$$\mathbb{E}[G_t \mathbf{h}_t] = \mathbb{E}[Q^{\pi}(s_t, a_t) \mathbf{h}_t].$$

这一步说明：REINFORCE 使用的 G_t 是 $Q^\pi(s_t, a_t)$ 的一次 Monte Carlo 样本；在期望意义上，可以把随机的 realized return 换成其条件平均 Q^π 。

接下来减去状态价值函数。附录 A.2 已经证明，只要 baseline 在给定状态后不依赖当前采样动作，就有式 (A.10)。令

$$b(s_t) = V^\pi(s_t),$$

则

$$\mathbb{E}[V^\pi(s_t)\mathbf{h}_t] = 0.$$

所以

$$\begin{aligned}\mathbb{E}[Q^\pi(s_t, a_t)\mathbf{h}_t] &= \mathbb{E}[(Q^\pi(s_t, a_t) - V^\pi(s_t))\mathbf{h}_t] \\ &= \mathbb{E}[A^\pi(s_t, a_t)\mathbf{h}_t].\end{aligned}$$

因此，在相同的 reward-to-go convention 下，可以把 Actor score 项的权重从

$$G_t \longrightarrow Q^\pi(s_t, a_t) \longrightarrow A^\pi(s_t, a_t).$$

这里需要区分三个层次： G_t 是一次 rollout 的随机样本； $Q^\pi(s_t, a_t)$ 是固定当前状态和动作后的条件平均； $A^\pi(s_t, a_t)$ 再减去状态平均水平 $V^\pi(s_t)$ ，表示当前动作相对于正常表现的优势。

与第二章的 convention 保持一致：当 $\gamma = 1$ 时，这些权重替换直接对应第一章未折扣目标的精确 Policy Gradient；当 $\gamma < 1$ 且第三、四章直接使用 discounted G_t^γ 时，它们定义的是本书采用的 discounted Actor 更新方向。若另行把目标从时间零定义为 $\mathbb{E}[\sum_t \gamma^t r_t]$ ，则严格梯度还需要保留相应的外层 γ^t 权重。

A.3.2 动作价值与状态价值的 Bellman 期望关系

从动作价值的定义开始：

$$Q^\pi(s_t, a_t) = \mathbb{E}_\pi[G_t \mid s_t, a_t].$$

Reward-to-Go 满足

$$G_t = r_t + \gamma G_{t+1}.$$

代入得到

$$Q^\pi(s_t, a_t) = \mathbb{E}[r_t + \gamma G_{t+1} \mid s_t, a_t].$$

关键在于处理未来项。严格地说，不能直接把 $\mathbb{E}[G_{t+1} \mid s_t, a_t]$ 机械改写成 $\mathbb{E}[V^\pi(s_{t+1}) \mid s_t, a_t]$ 而不说明条件结构。先把下一状态加入内部条件：

$$\begin{aligned} & \mathbb{E}[G_{t+1} \mid s_t, a_t] \\ &= \mathbb{E}\left[\mathbb{E}[G_{t+1} \mid s_t, a_t, s_{t+1}] \mid s_t, a_t\right]. \end{aligned} \quad (\text{A.13})$$

在 MDP 中，一旦给定 s_{t+1} ，从 $t+1$ 开始的未来演化不再需要过去的 (s_t, a_t) ；并且之后的动作按照当前策略 π 产生。因此

$$\mathbb{E}[G_{t+1} \mid s_t, a_t, s_{t+1}] = \mathbb{E}_\pi[G_{t+1} \mid s_{t+1}] = V^\pi(s_{t+1}). \quad (\text{A.14})$$

把式 (A.14) 代回式 (A.13)，得到

$$\mathbb{E}[G_{t+1} \mid s_t, a_t] = \mathbb{E}[V^\pi(s_{t+1}) \mid s_t, a_t].$$

令 $d_t = 1$ 表示这一步转移到 MDP 意义下的真正终止状态，否则 $d_t = 0$ 。对真正终止状态约定 $V^\pi(s_{t+1}) = 0$ ，因此可以等价地显式写出 **terminal mask**。最终，

$$Q^\pi(s_t, a_t) = \mathbb{E}[r_t + \gamma(1 - d_t)V^\pi(s_{t+1}) \mid s_t, a_t]. \quad (\text{A.15})$$

这种显式 **mask** 写法在函数近似实现中尤其重要，因为它不依赖网络在 **terminal** 输入上的任意输出，从而避免 **bootstrap** 穿过真正终止边界。

离散转移下的显式展开。 如果把给定 (s, a) 后所有可能的 **reward** 与下一状态写出来，则式 (A.15) 等价于

$$Q^\pi(s, a) = \sum_{s', r} P(s', r \mid s, a) [r + \gamma V^\pi(s')].$$

这条公式清楚地区分了单个分支值 $r + \gamma V^\pi(s')$ 和条件平均 $Q^\pi(s, a)$ 。前者对应某一个具体 (r, s') 分支的值，后者对所有可能分支按 $P(s', r \mid s, a)$ 做平均。当给定 (s, a) 后 **reward** 和下一状态都完全确定时，求和退化为单个分支；更一般地，只要所有正概率分支的合成值 $r + \gamma V^\pi(s')$ 相同，随机单步值也与其条件平均几乎处处相等。

A.3.3 TD Error 为什么可以估计 Advantage

Advantage 的定义是

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t).$$

沿用上一小节的终止标记 d_t ，式 (A.15) 已写成等价的 terminal-mask 形式。代入上式：

$$\begin{aligned} A^\pi(s_t, a_t) &= \mathbb{E}[r_t + \gamma(1 - d_t)V^\pi(s_{t+1}) \mid s_t, a_t] - V^\pi(s_t) \\ &= \mathbb{E}[r_t + \gamma(1 - d_t)V^\pi(s_{t+1}) - V^\pi(s_t) \mid s_t, a_t]. \end{aligned}$$

定义使用真实价值函数时的一步 TD error

$$\delta_t^\pi = r_t + \gamma(1 - d_t)V^\pi(s_{t+1}) - V^\pi(s_t),$$

于是

$$\mathbb{E}[\delta_t^\pi \mid s_t, a_t] = A^\pi(s_t, a_t).$$

因此，当 V^π 已知时，一次实际 transition 给出的 δ_t^π 是 Advantage 的单步随机样本；重复固定 (s_t, a_t) 并对所有可能的 reward、下一状态和后续未来取平均，才得到精确的 $A^\pi(s_t, a_t)$ 。

近似 Critic 带来的误差。实践中使用的是

$$V_\phi(s) \approx V^\pi(s),$$

因此实际 TD error 采用同一个 terminal mask：

$$\hat{\delta}_t = r_t + \gamma(1 - d_t)V_\phi(s_{t+1}) - V_\phi(s_t).$$

定义价值近似误差

$$e_\phi(s) = V_\phi(s) - V^\pi(s).$$

则直接相减得到

$$\hat{\delta}_t - \delta_t^\pi = \gamma(1 - d_t)e_\phi(s_{t+1}) - e_\phi(s_t).$$

对给定 (s_t, a_t) 取条件期望：

$$\begin{aligned}\mathbb{E} [\hat{\delta}_t \mid s_t, a_t] &= A^\pi(s_t, a_t) \\ &\quad + \gamma \mathbb{E} [(1 - d_t)e_\phi(s_{t+1}) \mid s_t, a_t] - e_\phi(s_t).\end{aligned}$$

所以，当 Critic 不完美时， $\hat{\delta}_t$ 一般不再是精确无偏的 Advantage 样本。这正是一步 Actor-Critic 用较低方差换取价值函数近似误差的来源之一。

需要区分这一结论与“把 $V_\phi(s_t)$ 仅作为 Monte Carlo baseline”时的情况。若 Actor 仍使用 $G_t - V_\phi(s_t)$ ，并且 baseline 在给定动作采样前历史后不依赖当前采样动作 a_t ，则根据附录 A.2 的零均值性质，减去这个 baseline 本身不会改变期望 Policy Gradient。真正引入额外近似的是用 $r_t + \gamma(1 - d_t)V_\phi(s_{t+1})$ 替代长未来回报进行 bootstrapping。

严格地说，在采用上述终止约定或等价 **terminal mask** 时， $r_t + \gamma(1 - d_t)V^\pi(s_{t+1})$ 是固定 (s_t, a_t) 后的一次随机单步结果， $Q^\pi(s_t, a_t)$ 是它的条件平均；同理， δ_t^π 是 Advantage 的单步随机样本，而 $A^\pi(s_t, a_t)$ 是其条件期望。实践中用近似 Critic V_ϕ 后，这个一步估计还会额外受到价值函数近似误差影响。

A.4 PPO 中的关键推导

第四章只保留 GAE 与 PPO-Clip 的主线。本附录集中展开四个在正文中会打断阅读节奏、但对理解公式边界很重要的问题：GAE 与加权 n -step Advantage 的等价关系、Probability Ratio 与 surrogate objective 的来源、PPO-Clipped Objective 的分段形式，以及新旧策略状态分布差异为什么使 PPO 只能有限度复用旧数据。

A.4.1 GAE 与加权 n -step Advantage 的等价关系

先考虑没有终止或 reset 的连续轨迹，并沿用正文的 discounted value convention；整段推导固定使用同一个 Critic。记一步 TD error 为

$$\hat{\delta}_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t).$$

正文已经给出

$$\hat{A}_t^{[n]} = \sum_{l=0}^{n-1} \gamma^l \hat{\delta}_{t+l}. \quad (\text{A.16})$$

下面说明为什么指数加权的 n -step Advantage 会化成 GAE 的 TD-error 形式。

对 $0 \leq \lambda < 1$ ，假定下述加权双重级数绝对可和，从而可以交换求和次序；例如 $\hat{\delta}_t$ 有界且 $\gamma\lambda < 1$ 是充分条件。定义

$$\hat{A}_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{[n]}. \quad (\text{A.17})$$

把式 (A.16) 代入：

$$\hat{A}_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \sum_{l=0}^{n-1} \gamma^l \hat{\delta}_{t+l}.$$

对固定的 l ，只有 $n \geq l + 1$ 的项会包含 $\hat{\delta}_{t+l}$ 。交换求和顺序：

$$\hat{A}_t^\lambda = \sum_{l=0}^{\infty} \gamma^l \hat{\delta}_{t+l} (1 - \lambda) \sum_{n=l+1}^{\infty} \lambda^{n-1}.$$

内部是几何级数：

$$(1 - \lambda) \sum_{n=l+1}^{\infty} \lambda^{n-1} = (1 - \lambda) \frac{\lambda^l}{1 - \lambda} = \lambda^l.$$

因此

$$\hat{A}_t^\lambda = \sum_{l=0}^{\infty} (\gamma\lambda)^l \hat{\delta}_{t+l}, \quad (\text{A.18})$$

这就是正文使用的 GAE 形式。

式 (A.17) 说明， λ 可以被理解为对不同 n -step Advantage 的指数混合；式 (A.18) 则说明，实现时可以直接把未来 TD error 做指数衰减累积。

当 $\lambda = 0$ 时，只剩 $n = 1$ 的一步估计； $\lambda = 1$ 不能直接代入含无限混合的式 (A.17)，而应使用满足边界条件的有限 TD 求和。

有限片段的尾权重与 bootstrap。 设从 t 起还有 $H \geq 1$ 条连续 transition，内部没有真正终止或 reset，末步可为真正终止或非终止截断。对 $1 \leq n \leq H$ ，使用同一个固定 Critic，并定义

$$G_t^{[n]} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n (1 - d_{t+n-1}) V_\phi(s_{t+n}),$$

$$\hat{A}_t^{[n]} = G_t^{[n]} - V_\phi(s_t).$$

相应的 TD residual 也使用 $1 - d_t$ 控制 bootstrap。令片段末尾之后的优势为零，则对 $0 \leq \lambda \leq 1$ ，有限 GAE 满足

$$\begin{aligned}\hat{A}_t^{\lambda, H} &= (1 - \lambda) \sum_{n=1}^{H-1} \lambda^{n-1} \hat{A}_t^{[n]} + \lambda^{H-1} \hat{A}_t^{[H]} \\ &= \sum_{l=0}^{H-1} (\gamma \lambda)^l \hat{\delta}_{t+l}.\end{aligned}$$

空和取零，零次幂取一。最后一个 H -step 项必须接收剩余权重 λ^{H-1} ，不能再乘 $1 - \lambda$ 。对第 l 个 TD residual，其系数中的混合权重为

$$(1 - \lambda) \sum_{n=l+1}^{H-1} \lambda^{n-1} + \lambda^{H-1} = \lambda^l,$$

故上述有限等价关系也覆盖 $H = 1$ 、 $\lambda = 0$ 与 $\lambda = 1$ 。特别地， $\lambda = 1$ 时

$$\hat{A}_t^{1, H} = \sum_{l=0}^{H-1} \gamma^l r_{t+l} + \gamma^H (1 - d_{t+H-1}) V_\phi(s_{t+H}) - V_\phi(s_t).$$

真正终止时端点 bootstrap 为零，得到完整 Monte Carlo return 减基线；仅截断时仍保留端点价值。实际 PPO 使用冻结的 V_{old} 。优势递推是否连接下一项由正文的 c_t 控制，而不是仅由真正终止标记 d_t 决定；这正是式 (4.3) 在 reset 与截断处仍有效的原因。

A.4.2 Probability Ratio 与 Surrogate Objective 的来源

PPO 的数据由 π_{old} 产生，但优化过程中希望评价 π_θ 。先固定一个已经采样到的状态 s ，只讨论动作分布的变化。

对任意函数 $f(s, a)$ ，若 $\pi_{\text{old}}(a | s) > 0$ 覆盖了新策略可能选择的动作，则

$$\begin{aligned}\mathbb{E}_{a \sim \pi_\theta(\cdot | s)}[f(s, a)] &= \sum_a \pi_\theta(a | s) f(s, a) \\ &= \sum_a \pi_{\text{old}}(a | s) \frac{\pi_\theta(a | s)}{\pi_{\text{old}}(a | s)} f(s, a) \\ &= \mathbb{E}_{a \sim \pi_{\text{old}}(\cdot | s)}[\rho(\theta; s, a) f(s, a)],\end{aligned}\tag{A.19}$$

其中

$$\rho(\boldsymbol{\theta}; s, a) = \frac{\pi_{\boldsymbol{\theta}}(a | s)}{\pi_{\text{old}}(a | s)}.$$

连续动作情形把求和换成积分，结构相同。

这里将 d^π 明确定义为给定数据收集规则下的**归一化**状态访问分布。对固定长度 T 的 rollout，它是各时间步状态分布的均匀混合。若收集独立的完整 episode、再将有效 transition 汇总平均，则相应的总体分布满足

$$d^\pi(\mathcal{B}) = \frac{\mathbb{E}_\pi[\sum_{t=0}^{T-1} \mathbb{I}_{\{s_t \in \mathcal{B}\}}]}{\mathbb{E}_\pi[T]}, \quad 0 < \mathbb{E}_\pi[T] < \infty,$$

其中 $\mathcal{B}$ 是可测状态集合， $\mathbb{I}$ 表示指示函数。这是按访问次数加权的分布；有限 batch 的经验平均不等于总体期望，也不自动构成原 episodic 求和目标的无偏估计。下面的 surrogate 始终固定使用旧策略及同一收集规则对应的 $d^{\pi_{\text{old}}}$ 。

如果把 $f(s, a)$ 取为旧策略下的 Advantage $A^{\pi_{\text{old}}}(s, a)$ ，并把 rollout 中的状态分布暂时固定为旧策略的状态访问分布 $d^{\pi_{\text{old}}}$ ，就得到局部 surrogate objective：

$$L^{\text{sur}}(\boldsymbol{\theta}) = \mathbb{E}_{\substack{s \sim d^{\pi_{\text{old}}} \\ a \sim \pi_{\text{old}}(\cdot | s)}} [\rho(\boldsymbol{\theta}; s, a) A^{\pi_{\text{old}}}(s, a)]. \quad (\text{A.20})$$

实践中真实 $A^{\pi_{\text{old}}}$ 未知，于是用 rollout 计算的 $\hat{A}_t$ 代替，得到正文中的 $\mathbb{E}_t[\rho_t(\boldsymbol{\theta}) \hat{A}_t]$ 。

这里必须注意：式 (A.19) 对固定状态下的动作分布是标准 Importance Sampling 恒等式；但式 (A.20) 把状态访问分布固定在 $d^{\pi_{\text{old}}}$ ，并没有把它自动变成新策略真实状态分布 $d^{\pi_{\boldsymbol{\theta}}}$ 。因此 PPO surrogate 是围绕旧策略数据建立的局部优化目标，而不是“任意新策略下真实长期回报”的无条件精确重写。

A.4.3 PPO-Clipped Objective 的分段形式

对单个样本，记 $A = \hat{A}_t$ 、 $\rho = \rho_t(\boldsymbol{\theta})$ ，其 clipped objective 为

$$\ell^{\text{CLIP}}(\rho, A) = \min(\rho A, \text{clip}(\rho, 1 - \epsilon, 1 + \epsilon) A).$$

当 $A \geq 0$ 。此时乘以 A 不改变大小关系。若 $\rho > 1 + \epsilon$ ，裁剪项更小；若 $\rho \leq 1 + \epsilon$ ，原始项不大于裁剪项。因此

$$\ell^{\text{CLIP}}(\rho, A) = \begin{cases} \rho A, & \rho \leq 1 + \epsilon, \\ (1 + \epsilon) A, & \rho > 1 + \epsilon. \end{cases} \quad A \geq 0.$$

也就是说，对正 Advantage 样本，PPO 截断的是“把该动作概率继续提高”所带来的额外收益；如果策略反而把好动作概率降得很低，目标仍然继续变差，并不会被下边界替它兜底。

当 $A < 0$ 。负数会反转乘法后的大小关系。若 $\rho < 1 - \epsilon$ ，裁剪后的 $(1 - \epsilon)A$ 更小，因此 $\min$ 选择裁剪项；若 $\rho \geq 1 - \epsilon$ ，选择原始项。于是

$$\ell^{\text{CLIP}}(\rho, A) = \begin{cases} (1 - \epsilon)A, & \rho < 1 - \epsilon, \\ \rho A, & \rho \geq 1 - \epsilon. \end{cases} \quad A < 0.$$

这时截断的是“继续把坏动作概率压低”所带来的额外收益；如果策略反而提高了坏动作概率，原始目标会继续变差，同样不会被上边界掩盖。

因此， $\min$ 与 clip 配合后的真正效果不是“无论如何都把 ratio 截回区间”，而是：只在样本已经沿正确方向走得过远时截断额外获益，同时保留错误方向更新带来的较差目标值。

A.4.4 新旧策略分布差异与 PPO 近似的边界

Probability ratio

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}$$

只比较同一个状态 s_t 上新旧策略给动作 a_t 的条件概率。但完整策略目标还依赖策略会访问哪些状态。

把时间步吸收到状态访问分布中，一个 Advantage 形式的策略目标会同时涉及状态访问分布 $d^\pi(s)$ 和动作分布 $\pi(a | s)$ 。旧策略 rollout 给出的状态来自 $s \sim d^{\pi_{\text{old}}}$ ，而真正运行新策略时，状态来自 $s \sim d^{\pi_\theta}$ 。即使在某个已访问状态上用 ratio 精确修正了动作分布，也没有因此自动修正未来状态访问分布的变化。

这解释了为什么“新旧策略足够接近”对 PPO 很重要。如果动作分布只发生局部变化，状态访问分布通常也不会立刻完全改变，此时围绕旧策略构造的 surrogate objective 更有参考价值；若策略一次移动很远，旧 rollout 对新策略的代表性会快速下降。

TRPO 把这种思想写成显式的 trust-region 约束；PPO-Clip 则采用更容易通过一阶优化实现的 surrogate clipping。需要注意，PPO-Clip 不是严格的 KL 约束，也不

保证每个样本的 ratio 永远停在 $[1 - \epsilon, 1 + \epsilon]$ 内。它只是通过目标函数设计，减少某些已经变化过大的样本继续推动策略远离旧策略的优化激励。

因此，PPO 的多 epoch 数据复用有一个天然边界：

旧数据越重复使用 $\implies \pi_\theta$ 通常越远离 $\pi_{\text{old}} \implies$ surrogate 近似越需要谨慎.

这也是 PPO 训练若干 epoch 后重新 rollout，而不是把旧数据无限保留并长期训练的理论直觉。